VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI– 590018, KARNATAKA, INDIA

A PROJECT REPORT

on

"Swaraaha: Stutter Classification and Localization System"

Submitted in partial fulfilment of the requirements for the award of

BACHELOR OF ENGINEERING

in

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Submitted By

Name	USN
K SHREEKRISHNA UPADHYAYA	4VP23AI020
M CHETHAN KESHAV BHAT	4VP23AI023
SKANDA PRASAD K	4VP23AI051
SRINIVAS HEGDE M	4VP23AI054

Under the Guidance of

Prof. Ajay Shastry C G

Assistant Professor

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

VIVEKANANDA COLLEGE OF ENGINEERING & TECHNOLOGY

[A Unit of Vivekananda Vidyavardhaka Sangha Puttur (R)]

Affiliated to Visvesvaraya Technological University and Approved by AICTE New Delhi & Govt., of Karnataka

Nehru Nagar, Puttur - 574 203, DK, Karnataka, India.

November, 2026

VIVEKANANDA COLLEGE OF ENGINEERING & TECHNOLOGY

[A Unit of Vivekananda Vidyavardhaka Sangha Puttur (R)]

Affiliated to Visvesvaraya Technological University and Approved by AICTE New Delhi & Govt. of Karnataka

Nehru Nagar, Puttur - 574203, DK, Karnataka, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CERTIFICATE

Certified that the project work entitled **"Swaraaha: Stutter Classification and Localization System"** is carried out by **Mr. K Shreekrishna Upadhyaya, Mr. M Chethan Keshav Bhat, Mr. Skanda Prasad K, Mr. Srinivas Hegde M** bearing USNs **4VP23AI020**, **4VP23AI023, 4VP23AI051 and 4VP23AI054** respectively bonafide students of **Vivekananda College of Engineering & Technology, Puttur** in partial fulfilment for the award of **Bachelor of Engineering** in **Artificial Intelligence & Machine Learning** of the **Visvesvaraya Technological University**, Belagavi during the year 2026-27. It is certified that all corrections/suggestions indicated during Internal Assessment have been incorporated in the report deposited in the departmental library.

The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Signature of the Guide	Signature of the HOD	Signature of the Principal
Prof. Ajay Shastry C G	**Dr. Radhika Shetty D S**	**Dr. Purushothama Chippar**

EXTERNAL VIVA

Name of the Examiners	Signature with date
1…….....	.....
2…….....	.....

DECLARATION

We, **Mr. K Shreekrishna Upadhyaya (4VP23AI020), Mr. M Chethan Keshav Bhat (4VP23AI023), Mr. Skanda Prasad K (4VP23AI051), Mr. Srinivas Hegde M (4VP23AI054)** students of B.E. 7th Semester in Artificial Intelligence & Machine Learning, **Vivekananda College of Engineering & Technology**, Puttur, hereby declare that the project work entitled **"Swaraaha: Stutter Classification and Localization System"** has been carried out by us at VCET, Puttur, under the guidance of **Prof. Ajay Shastry C G,** Assistant Professor, Department of Artificial Intelligence & Machine Learning, Vivekananda College of Engineering & Technology, Puttur, and submitted in partial fulfilment of the requirements for the award of degree in **Bachelor of Engineering in Artificial Intelligence & Machine Learning** by **Visvesvaraya Technological University**, Belagavi during the academic year 2026-2027.

Name of the students	USN	Signature with date
K Shreekrishna Upadhyaya	4VP23AI020	
M Chethan Keshav Bhat	4VP23AI023	
Skanda Prasad K	4VP23AI051	
Srinivas Hegde M	4VP23AI054	

Date:

Place: Puttur

ACKNOWLEDGEMENT

We take this opportunity to express our deep heartfelt gratitude to all those people who have helped us in the successful completion of the project.

First and foremost, we would like to express our sincere gratitude to our guide, **Prof. Ajay Shastry C G.** for providing excellent guidance, encouragement and inspiration throughout the project work. Without his invaluable guidance, this work would never have been a successful one.

We would like to express our sincere gratitude to Head of the Department of Artificial Intelligence & Machine Learning, **Dr. Radhika Shetty D S** for her guidance and inspiration.

We would like to thank our Principal, **Dr. Purushothama Chippar.** for providing all the facilities and a proper environment to work in the college campus.

We would like to thank our management for providing the necessary infrastructure to carry out the project work.

We are thankful to all the teaching and non-teaching staff members of Artificial Intelligence & Machine Learning Department for their help and needed support rendered throughout the project.

ABSTRACT

Stuttering disrupts the natural flow of communication through involuntary repetitions, prolongations, blocks, and pauses. Roughly 1% of people worldwide are affected, and the condition can reach into their social, professional, and personal life. Diagnosis today still relies mostly on manual assessment by speech-language pathologists, which is subjective, time-consuming, and not available to everyone. This project builds an automated system that uses deep learning to detect, classify, and analyze stuttering events in speech recordings. The system identifies the types of dysfluencies present and reports when they occur. Users record or upload audio through a simple interface and receive the results in an accessible format. The result is a tool for early stutter detection and objective speech evaluation, useful to both speech-language pathologists and people in treatment. The system also contributes to the growing use of artificial intelligence in healthcare, supporting diagnosis, therapy, and an improved quality of life for people who stutter.

TABLE OF CONTENTS

Title	Page No.
LIST OF TABLES	I
LIST OF FIGURES	II
LIST OF ABBREVIATIONS	III
CHAPTER 1 INTRODUCTION	**01**
1.1 INTRODUCTION TO THE PROJECT	01
1.2 PROPOSED METHOD	01
1.3 LITERATURE SURVEY	02
1.4 PROBLEM STATEMENT	14
1.5 OBJECTIVES	14
CHAPTER 2 REQUIREMENT SPECIFICATION AND ANALYSIS	**16**
2.1 INTRODUCTION	16
2.2 FUNCTIONAL REQUIREMENTS	16
2.2.1 User Requirements	17
2.2.2 System Requirements	17
2.3 NON-FUNCTIONAL REQUIREMENTS	17
2.3.1 Performance	18
2.3.2 Scalability	18
2.3.3 Availability	18
2.3.4 Usability	18
2.4 Software Requirements	19
2.5 Hardware Requirements	19
CHAPTER 3 SYSTEM DESIGN	**21**
3.1 INTRODUCTION	21
3.2 SYSTEM ARCHITECTURE AND DESIGN	21
3.3 FLOWCHART	24
3.4 USE CASE DIAGRAM AND DESCRIPTIONS	27
3.5 WORKFLOW DIAGRAM	29
3.6 SEQUENCE DIAGRAM	30
CHAPTER 4 SYSTEM IMPLEMENTATION	**33**
4.1 INTRODUCTION	33
4.2 ALGORITHM	33
4.3 IMPLEMENTATION REQUIREMENTS	34
4.3.1 Train the Model	35

4.3.2 Test the Model 35
4.3.3 Model Evaluation 36
4.4 USER INTERFACE 37
CHAPTER 5 SYSTEM TESTING 39
5.1 INTRODUCTION 39
5.2 TYPES OF TESTS 39
5.2.1 Unit Testing 39
5.2.2 Integration Testing 40
5.2.3 Functional Testing 40
5.2.4 System Testing 40
5.3 RESULT AND ANALYSIS 41
5.3.1 Model Evaluation of the Wav2Vec2 Classifiers 41
5.3.2 Wav2Vec2 Classifier Training 45
5.3.3 Model Evaluation of the CNN-based Models 46
5.3.4 CNN-based Model Training 47
5.3.5 Hyper Parameter Tuning 47
5.3.6 Performance Analysis of the Models 48
CHAPTER 6 SCREENSHOTS 50
CHAPTER 7 CONCLUSION AND SCOPE FOR FUTURE ENHANCEMENTS 53
7.1 CONCLUSION 53
7.2 SCOPE FOR FUTURE ENHANCEMENTS 53
REFERENCES 55
PERSONAL PROFILE 58

LIST OF TABLES

Table No.	Title	Page No.
1.1	Summary of Literature Survey	09
5.1	Multitask Wav2Vec2 classifier results	41
5.2	Wav2Vec2 localizer evaluation results	44
5.3	Threshold optimization results	47
5.4	Comparative study of seven architecture variants	48

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	System Architecture	22
3.2	Flowchart	26
3.3	Use Case Diagram	28
3.4	Workflow Diagram	29
3.5	Sequence Diagram	31
5.1	Per-class F1 of the multitask Wav2Vec2 classifier	42
5.2	Per-class AUROC and AUPRC of the multitask Wav2Vec2 classifier	42
5.3	Confusion matrices for the five binary classifiers	43
5.4	Frame-level metrics of the Wav2Vec2 localizer	45
5.5	Per-class tuned F1 on the Boli set	49
6.1	Home page of the web interface	50
6.2	Results page of the web interface	50
6.3	Home page of the desktop application	51
6.4	Analysis result page of the desktop application	52

LIST OF ABBREVIATIONS

Abbreviation	Description
ASR	Automatic Speech Recognition
AUROC	Area Under the Receiver Operating Characteristic Curve
AUPRC	Area Under the Precision-Recall Curve
BERT	Bidirectional Encoder Representations from Transformers
Bi-LSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
CTC	Connectionist Temporal Classification
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HMM	Hidden Markov Model
HuBERT	Hidden-Unit BERT
isWER	Intended Speech Word Error Rate
LLM	Large Language Model
LPC	Linear Predictive Coding
LSTM	Long Short-Term Memory
MFCC	Mel-Frequency Cepstral Coefficients
PCA	Principal Component Analysis
PWS	People Who Stutter
RAG	Retrieval-Augmented Generation
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
SEP-28K	Stuttering Event Detection Dataset (28,000 samples)
SLP	Speech-Language Pathologist
SOTA	State of the Art
SVM	Support Vector Machine
TTS	Text-to-Speech
UAR	Unweighted Average Recall
UI	User Interface
VITS	Variational Inference with Adversarial Learning for end-to-end Text-to-Speech
WavLM	Wave Language Model
WER	Word Error Rate
YOLO	You Only Look Once

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION TO THE PROJECT

Speech fluency lets a person express thoughts and feelings clearly. Stuttering is a common speech disorder that interrupts speech production with involuntary repetitions, prolongations, or blocks. It affects people of all ages, and it can wear down confidence and make communication and social interactions harder. Spotting stuttering patterns early makes therapy and management easier.

Detection and assessment usually depend on speech-language pathologists watching and listening manually. Results can be subjective, slow, and inconsistent from one expert to another, and the work takes a physical toll on the pathologist. That reliance on human evaluation makes large-scale assessment and continuous monitoring difficult. Recent advances in deep learning and speech signal processing have made automated stutter detection a reliable, scalable alternative. This project builds a deep learning system that automatically detects, classifies, and analyzes five stuttering types: prolongations, blocks, sound repetitions, word repetitions, and interjections.

Input audio is preprocessed and converted into embeddings, then analyzed with Wav2Vec 2.0, which captures fine acoustic detail and dependencies across longer stretches of speech that separate fluent speech from dysfluent speech. Alongside classification, the same audio passes through a localization module that returns the timestamps of the stuttering events. Those timestamps are hard-aligned with the transcript timestamps, so the system flags only the affected speech segments. The finished model sits behind a simple interface where users record or upload speech samples and receive an analysis with time-stamped detection results and classification output.

The project aims to give speech-language pathologists and people who stutter an accurate, objective, and accessible diagnostic tool. The system also contributes to the growing use of artificial intelligence in healthcare, supporting diagnosis, therapy, and an improved quality of life for people who stutter.

1.2 PROPOSED METHOD

The proposed method aims to develop an efficient deep learning-based system for the early detection and classification of speech dysfluencies from audio recordings. The process begins with comprehensive audio preprocessing that includes noise reduction,

normalization, and enhancement of acoustic clarity across all samples. Segmentation isolates the meaningful regions of speech, eliminating irrelevant silence and background, and data augmentation such as noise injection, time and pitch shifting, and scaling simulates variations in speaking rate and recording conditions. The system accurately categorizes the input into five dysfluency types: prolongation, block, sound repetition, word repetition, and interjection, and localizes the segments where these occur, supporting early diagnosis and effective treatment planning. This automated approach reduces manual workload, minimizes observer bias, and enhances diagnostic reliability, making it a valuable tool in computer-aided speech assessment.

1.3 LITERATURE SURVEY

P. Arbajian et al. [1] A speech segment selection strategy for stutter block detection was studied in this paper. The authors extract acoustic features from annotated speech samples and test machine learning classifiers under different segmentation setups, comparing windowing approaches. The results show that segment choice strongly affects accuracy: some segment lengths model dysfluency patterns better, while bad segmentation drops important temporal information and degrades performance. Segmentation choices therefore matter for detection accuracy and for remediation systems, which the proposed system takes into account in its preprocessing.

V. Mitra et al. [2] This study developed a tuning-based approach that makes a voice assistant handle dysfluent speech better by optimizing an existing hybrid ASR system. The goal is to reduce recognition errors caused by stuttering, such as repetitions and unintended insertions. The method changes the key decoding parameters of the ASR system, giving more weight to the language model by raising the penalty for inserted words and lowering the acoustic model weight. That helps the system filter repeated or disfluent segments and focus on meaningful speech. Tested on speech from 18 participants with varying stuttering severity, it reduced the intended speech Word Error Rate (isWER) by 24% relative. The results show that parameter tuning alone can reduce dysfluency errors without restructuring the ASR architecture, which is why the proposed system keeps model tuning and post-processing in mind.

P. Mohapatra et al. [3] disfluencyNet, a deep learning model that detects speech disfluencies using contextual representations, was introduced in this paper. The model feeds contextual embeddings into a classification network, modeling speech dependencies to catch disfluencies such as repetitions and pauses. Data distillation limits the need for large training sets. Trained and evaluated on SEP-28k and FluencyBank, the model matched

baseline accuracy using only a quarter of the data, which points to efficient training and strong generalization. That supports the proposed system's use of contextual embeddings and efficient training where annotated data is scarce.

J. Liu et al. [4] A method for detecting disfluencies in multilingual speech of varying lengths was proposed in this paper using the context-based embeddings of wav2vec 2.0. A classification network takes the embeddings, and data distillation keeps only high-quality audio fragments where three human annotators agree on the disfluency labels. Tests on multilingual datasets with different speech lengths showed better performance than other approaches. The work supports the proposed system's use of pretrained models like wav2vec 2.0 and data filtering to improve robustness and cross-speaker generalization.

A. Romana et al. [5] The study introduced a multimodal approach that detects speech disfluencies directly from untranscribed audio. A Bi-LSTM fusion model runs at the frame level, combining WavLM acoustic features with BERT language representations drawn from transcripts produced by a fine-tuned Whisper model. This captures both low-level speech patterns and high-level context, and it copes with ASR transcription errors and misalignment. The multimodal setup proved more robust and accurate than single-modality approaches in noisy real-world conditions. That supports combining acoustic and language features in the proposed system.

D. Wagner et al. [6] This paper presented a hybrid approach that uses Large Language Models (LLMs) to combine acoustic and linguistic representations for dysfluency detection. Acoustic features come from wav2vec 2.0 and transcriptions from Whisper ASR; the two are fused into a joint input that an LLM classifies into stutter types including repetitions, prolongations, and blocks. The acoustic-lexical combination outperformed models that use only audio or only text. The result backs the proposed system's plan to combine audio and language signals for better accuracy and generalization.

S. A. Sheikh et al. [7] The study developed a deep learning framework called multi-contextual (MC) StutterNet for stuttering detection that tackles class imbalance and limited data. The multi-branch architecture processes different contextual representations of speech. Class-balanced loss assigns more weight to underrepresented stutter categories, and data augmentation diversifies the training set. Results showed more robust and accurate detection across speech conditions, especially for less frequent dysfluency types. The proposed system uses the same ideas: data augmentation, balanced loss, and multi-context learning.

X. Zhou et al. [8] A YOLO-inspired model that treats spectrograms as images was introduced in this paper, detecting dysfluencies with simultaneous localization and classification in the time-frequency domain. A region-based detection model learns spatial features (frequency patterns) and temporal dynamics (changes over time) from these images. Speech-text alignment links audio segments to their corresponding words for better context. The model predicts both the dysfluency type and the exact time region where it occurs, with strong results across multiple dysfluency types. This is why the proposed system builds its real-time, region-wise detection on spectrogram-based CNN architectures.

X. Zhou et al. [9] stutter-Solver, an end-to-end YOLO-inspired model for multilingual dysfluency detection, was developed in this paper. It treats speech spectrograms as image-like inputs and applies a region-based detection framework. To address data scarcity, the authors generated synthetic datasets (VCTK-Pro, VCTK-Art, AISHELL3-Pro) using articulatory and text-to-speech (TTS) simulations. The model performed well across multiple datasets and languages, and the work supports the proposed system's multilingual capability and use of synthetic data.

J. Zhang et al. [10] This paper proposed LLM-Dys, a method for generating large-scale dysfluent speech datasets using Large Language Models (LLMs). An LLM simulates realistic dysfluency patterns and produces labeled dysfluent text, which the VITS model converts into synthetic audio. Unlike rule-based methods, the generated speech keeps more natural prosody and contextual variation. The dataset covers 11 categories of dysfluencies at both word and phoneme levels, enabling fine-grained analysis and model training. The resulting models performed better on detection tasks, which backs the proposed system's use of LLMs and TTS for data augmentation.

S. Kim and A. Kumar [11] FluentNet, a hybrid CNN-LSTM architecture for end-to-end speech disfluency detection, was developed in this paper. CNN layers extract short-term spectral features from mel-spectrograms, and LSTM layers model temporal continuity, which suits recurring stutter patterns. Trained and validated on SEP-28k, the model passed 91% classification accuracy. It also ran in real time in speech therapy, giving users visual dysfluency feedback within a feedback loop. End-to-end models that skip handcrafted features reduce bias and generalize across speakers and environments, and the architecture informs the multi-model CNN approach in the proposed system.

R. Ahmed and J. Park [12] This study developed a multilingual speech dysfluency detection framework that identifies stuttering events across English, Korean, and Japanese datasets. Built on an encoder-decoder transformer architecture similar to BERT, the model captures contextual relationships in speech sequences. Trained on combined multilingual datasets, it showed high adaptability to linguistic variation, achieving an F1-score of 95.2% in English and over 93% in non-English corpora. Attention visualization showed which segments of the input most influenced the detection decision. The work supports the proposed system's cross-language generalization and interpretability in dysfluency detection.

F. Rahimi and D. Torres [13] This research presented an exploration of large language models (LLMs) and transformer-based architectures for speech dysfluency analysis. Speech features are embedded as tokenized sequences so the model can detect contextual disruptions in spoken language. The LLM-based approach outperformed CNN and RNN models in both recall and precision, particularly for subtle dysfluency types like interjections and soft blocks. Pretraining on large general speech datasets also helped on smaller stutter-specific corpora, and attention visualization made the predictions transparent. The findings point to combining CNN-based acoustic analysis with language-level context in the proposed system.

V. Uloza et al. [14] An AI-based algorithm for assessing pathological speech, specifically substitution voicing, was developed in this paper using deep neural networks. CNNs and principal component analysis (PCA) perform the classification, with preprocessing such as noise removal and normalization giving the networks consistent input. Classification accuracy exceeded 93%, confirming that AI can detect subtle speech impairments. Though the study targets substitution voicing rather than stuttering, it offers useful lessons for speech pathology systems built on spectral analysis, which is the basis for the proposed system's use of mel-spectrograms and CNN feature extraction.

H. Müller and C. Lee [15] This study analyzed the neural bases of speech production in people who stutter through a meta-analysis of fMRI and EEG studies. Identified regions include the inferior frontal gyrus and basal ganglia, which show atypical activation during speech production. These findings provide biological grounding for AI-based stutter detection, since the acoustic signatures such systems rely on reflect underlying differences in neural control. The study proposes no computational model, but it links the speech irregularities the proposed system detects to their neurological causes, which is useful for feature selection and interpretation.

A. Baevski et al. [16] wav2vec 2.0 was introduced in this paper as a self-supervised framework for learning speech representations directly from raw audio waveforms. A convolutional feature encoder extracts latent representations that a transformer encoder then contextualizes. Pretrained on 960 hours of unlabeled LibriSpeech data and fine-tuned with a CTC loss, the model performed strongly on phoneme recognition, speaker identification, and emotion recognition benchmarks. This is the foundational speech representation model in the proposed system, making feature extraction from stuttered speech possible without large labeled datasets.

P. Khanna et al. [17] The study presented StuD, a multimodal stuttering detection system that combines acoustic features from Wav2Vec 2.0 and HuBERT with linguistic features from Llama-2, enhanced by Retrieval-Augmented Generation for adaptive classification. A fusion strategy weights the acoustic and linguistic embeddings by input quality. The RAG component retrieves similar historical cases to improve classification of rare stuttering patterns. Evaluated on SEP-28k and FluencyBank, the system performed strongly across all stuttering event types. This multimodal fusion approach supports the proposed system's goal of combining audio and text features for comprehensive stutter analysis.

C. Lea and V. Mitra [18] SEP-28K, a dataset of over 28,000 speech clips drawn from stuttering support group podcasts and annotated for five stuttering event types (blocks, prolongations, repetitions, interjections, and revisions), was introduced in this paper. The clips carry both crowdsourced annotations from non-expert listeners and expert annotations from speech-language pathologists. The authors report that scaling annotation from 10k to 28k clips improved F1 by 28% and 24% for blocks and prolongations respectively. The dataset addresses the data scarcity problem in stuttering research and is the primary benchmark for evaluating the proposed detection system.

O. Shonibare et al. [19] This paper proposed a two-stage approach for automatic speech recognition on stuttered speech with limited labeled data. A context-aware classifier trained on small amounts of labeled stuttering data detects dysfluency regions. A modified ASR decoder then passes over the detected dysfluencies, focusing transcription on fluent segments. The method cut word error rate by 12-71% across different stuttering severity levels. The detect-then-transcribe design shows that dysfluency detection and speech processing can be decoupled, which is the premise of the proposed system's modular architecture.

S. Bayerl et al. [20] The study applied fine-tuned wav2vec 2.0 to dysfluency detection in clinical stuttering therapy recordings. Acoustic embeddings are combined with a multi-task learning framework and an SVM classifier to distinguish dysfluency types in therapy speech. Evaluated on FluencyBank and the German KSoF dataset, it improved F1 by 27% over baseline methods. The multi-task approach predicts dysfluency presence and type together, letting therapists track progress over sessions. The clinical context confirms that pretrained speech models transfer to real-world stuttering assessment.

S. Bayerl et al. [21] This paper introduced a modified wav2vec 2.0 framework that treats stuttering detection as a multi-label classification problem, on the grounds that dysfluencies often occur together in natural speech. The model assigns an independent probability score to each dysfluency type, so multiple overlapping events are detected at once. It performed well on SEP-28k-Extended and generalized from English to German. The multi-label formulation fits the reality of stuttering, where a single utterance can contain blocks, prolongations, and repetitions at the same time.

R. Gong et al. [22] AS-70, the first and largest Mandarin stuttered speech dataset, was introduced in this paper, containing 70 hours of recordings from speakers at varying stuttering severity levels. The corpus covers diverse demographics (age groups, genders, and stuttering types) alongside word- and syllable-level dysfluency labels. The dataset is released as an open source resource to encourage Mandarin stuttering research. It extends stuttering data beyond English and enables the development of multilingual systems.

X. Liu et al. [23] An end-to-end stuttering detection architecture was introduced in this paper, combining Conformer blocks for local acoustic pattern extraction with BiLSTM layers for long-range temporal dependency modeling. The multi-task framework predicts dysfluency types and severity levels together. It took first place in the SLT 2024 Stuttering Detection Challenge and improved F1 by 39.8% on the AS-70 Mandarin dataset. Conformer's mix of convolution and self-attention captures both local and global speech patterns, which proved effective for the varied temporal signatures of different stuttering events.

A. Batra et al. [24] Boli, a multilingual Indian language dataset for understanding stuttering including both read and spontaneous speech, was introduced in this paper. The corpus captures real-world stuttering patterns from Indian speakers across five stuttering types. The dataset includes demographic metadata and self-reported stuttering severity ratings. It addresses the gap in non-English stuttering datasets, particularly for South Asian

languages where stuttering prevalence and manifestation patterns differ from Western populations.

A. R. Valente et al. [25] This study introduced an enhanced version of the FluencyBank dataset with detailed clinical annotations provided by expert speech-language pathologists. The annotations cover audiovisual cues and secondary stuttering behaviors such as facial tension and eye blinking, plus physiological tension indicators, beyond the usual acoustic dysfluency labels. The multi-dimensional scheme captures more of the clinical picture of stuttering severity. Such expert annotations make it possible to train detection models that account for the full stuttering experience rather than acoustic events alone.

T. Grósz et al. [26] This paper presented a wav2vec2-based paralinguistic analysis system that jointly recognizes vocalized emotions and stuttering events from speech. Self-supervised speech embeddings feed task-specific classification heads for both tasks. The system reached 62.1% unweighted average recall on the Stuttering Sub-Challenge benchmark. The results connect emotional state and speech disfluency, and suggest that modeling the two together improves detection through shared representations.

J. Tang et al. [27] This paper presented comprehensive speech annotation guidelines developed in collaboration with people who stutter, standardizing how stuttering events are labeled in speech datasets. The guidelines resolve the hard cases in annotating ambiguous dysfluencies with clear categorical definitions and decision trees. They also cover annotator training, inter-annotator agreement measurement, and handling edge cases. Community-informed guidelines raise annotation quality and reproducibility across stuttering research, which directly helps dataset construction and model evaluation.

A. Romana et al. [28] The study introduced an updated version of the FluencyBank dataset with word-level timestamps and refined disfluency annotations. Updated transcripts carry granular disfluency labels and word timing for each clip, letting speech processing models be measured on disfluent input. The resource supports word-level dysfluency localization in stuttering research, which is exactly what the proposed system needs to map events to the words where they occur.

L. Nie et al. [29] MMSD-Net, the first multimodal neural framework for stuttering detection combining audio and visual signals through transformer-based cross-modal fusion, was introduced in this paper. The model processes speech embeddings alongside facial expression and lip movement features to capture both the acoustic and visual

manifestations of stuttering. It improved F1 by 2-17% over single-modality approaches on benchmark datasets. Visual information clearly helps, especially for events like blocks and prolongations that have distinct visual signatures.

R. P. Buzzeti et al. [30] This paper proposed a two-stage hybrid approach for detecting and classifying stuttering-related disfluencies in Brazilian Portuguese. The first stage identifies potential disfluency regions from linguistic rules, and the second performs severity assessment. Portuguese stuttering differs from English in syllable structure and how dysfluencies manifest, and the method addresses those language-specific patterns. It extends stuttering detection to an underrepresented language and shows that hybrid approaches combining linguistic rules with data-driven classification work.

The studies above cover acoustic classifiers, self-supervised speech representations, multimodal fusion, and synthetic data generation. Table 1.1 compares them by features, strengths, and limitations. The field has moved from hand-crafted acoustic features toward large pre-trained models, while multilingual coverage, localization, and clinical reliability remain open.

Sl. No	Author	Title	Features	Pros	Cons
1	Pierre Arbajian et al. [1]	Effect of speech segment samples selection in stutter block detection and remediation	Analyzed different speech segment lengths and sample selection methods for accurate stutter block detection using acoustic classifiers.	Improves detection precision through better segment selection.	Sensitive to segmentation configuration.
2	Vikramjit Mitra et al. [2]	Analysis and Tuning of a Voice Assistant System for Dysfluent Speech	Modified ASR decoding parameters by increasing word insertion penalty and reducing acoustic model influence to suppress repetition errors	Enhances intended speech recognition for dysfluent users.	Focuses on recognition rather than dysfluency classification.
3	Payal Mohapatra et al. [3]	Speech Disfluency Detection with Contextual Representation and Data Distillation	Developed DisfluencyNet using contextual embeddings and distilled high-confidence samples	Achieves strong results with reduced training data	Depends on carefully filtered annotations

Sl. No	Author	Title	Features	Pros	Cons
			for efficient low-resource training		
4	Jiajun Liu et al. [4]	Automatic Speech Disfluency Detection Using Wav2Vec 2.0 for Different Languages	Applied Wav2Vec 2.0 contextual embeddings for multilingual disfluency detection across variable speech durations.	Supports multilingual detection with strong contextual learning.	High resource requirements and limited availability of multilingual stuttering datasets.
5	Amrit Romana et al. [5]	Automatic Disfluency Detection from Untranscribed Speech	Built multimodal Bi-LSTM combining WavLM acoustic signals with BERT linguistic features from Whisper transcripts.	Detects disfluencies without manual transcription.	Multimodal design increases system complexity.
6	Dominik Wagner et al. [6]	Large Language Models for Dysfluency Detection in Stuttered Speech	Combined Wave2Vec 2.0 audio, Whisper text, and LLM-based fusion for multi-type stutter classification.	Improves multi-class detection accuracy.	Requires heavy computational resources.
7	Shakeel A. Sheikh et al. [7]	Advancing Stutter Detection via Data Augmentation, Class-Balanced Loss and Multi-Contextual Deep Learning	Proposed multi-contextual StutterNet with augmentation and weighted loss for robust stuttering detection.	Addresses data scarcity and class imbalance.	Synthetic augmentation may reduce realism.
8	X. Zhou et al. [8]	YOLO-Stutter: End-to-End Region-Wise Speech Dysfluency Detection	Adapted YOLO on speech spectrograms for simultaneous dysfluency type prediction and temporal localization.	Enables precise real-time stutter localization.	Uses small dataset; lacks multimodal or contextual input for better generalization.
9	Xuanru Zhou et al. [9]	Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection	Developed multilingual YOLO-based dysfluency detector using synthetic articulatory and TTS-generated datasets.	Expands multilingual coverage with SOTA accuracy.	Synthetic speech may not fully match real speech.
10	Jinning Zhang et al. [10]	Analysis and Evaluation of Synthetic Data Generation in	Proposed LLM-Dys framework using LLM-generated dys-	Generates diverse large-scale dysfluency datasets.	Synthetic prosody may limit robustness.

Sl. No	Author	Title	Features	Pros	Cons
		Speech Dysfluency Detection	fluent text and VITS TTS for scalable corpus creation.		
11	S. Kim & A. Kumar [11]	FluentNet: End-to-End Detection of Speech Disfluency with Deep Learning	CNN-LSTM hybrid model analyzing temporal and spectral dependencies in speech signals using SEP-28k dataset.	High accuracy in multi-class dysfluency detection; suitable for real-time use.	Model complexity increases training time and requires GPU-based systems.
12	R. Ahmed & J Park [12]	Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection	Transformer-based multilingual model for detecting stuttering across multiple languages using contextual embeddings.	Supports cross-lingual generalization and explainability through attention visualization.	High resource requirements and limited availability of multilingual stuttering datasets.
13	F. Rahimi & D. Torres [13]	Large Language Models for Dysfluency Detection in Stuttered Speech	Used transformer-based large language models (LLMs) to capture context disruptions in stuttered speech	Outperforms CNN and RNN baselines on subtle dysfluency types.	Requires heavy computational resources.
14	V. Uloza et al. [14]	An AI-Based Algorithm for the Assessment of Substitution Voicing	CNN and PCA combine feature extraction and classification of pathological voice disorders.	Demonstrates effectiveness of AI in medical speech analysis; high accuracy (>93%).	Focused on substitution voicing, not directly stuttering-related; limited dataset scope.
15	H Muller & C. Lee [15]	Reinvestigating the Neural Bases Involved in Speech Production of Stutterers: An ALE Meta-Analysis	Analyzed fMRI/ EEG studies to identify brain regions linked to speech dysfluency.	Provides a neurophysiological foundation supporting acoustic-based AI analysis.	Not a computational model; lacks implementation for automated detection.
16	A. Baevski et al. [16]	wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations	Self-supervised pretraining of speech representations using masked prediction with transformer and convolutional encoder on raw audio.	Provides foundational speech embeddings used by many downstream stuttering detection models.	Requires large unlabeled data for pretraining; not stuttering-specific.
17	P. Khanna et al. [17]	StuD: A Multi-modal Approach for	Combined Wav2Vec 2.0, HuBERT	Achieves SOTA on SEP-28k and Fluen-	Heavy computational requirements

Sl. No	Author	Title	Features	Pros	Cons
		Stuttering Detection with RAG and Fusion Strategies	acoustic features with Llama-2 linguistic features and RAG-based adaptive classification.	cyBank with adaptive retrieval.	for LLM and RAG components.
18	C. Lea and V. Mitra [18]	SEP-28K: A Dataset for Stuttering Event Detection from Podcasts	Large-scale dataset with 28k+ clips annotated for 5 stuttering event types from podcast recordings.	Enables large-scale training with 28%/24% F1 gains from data scaling.	Podcast speech may not generalize to clinical or spontaneous settings.
19	O. Shonibare et al. [19]	Enhancing ASR for Stuttered Speech with Limited Data using Detect and Pass	Two-stage approach: context-aware dysfluency detection followed by ASR that passes over detected events.	Achieves 12-71% WER reduction with minimal labeled data.	Relies on accurate first-stage detection; cascaded errors possible.
20	S. Bayerl et al. [20]	Detecting Dysfluencies in Stuttering Therapy Using wav2vec 2.0	Fine-tuned wav2vec 2.0 with multi-task learning and SVM for therapy speech analysis.	27% F1 improvement on clinical therapy recordings.	Limited to therapy contexts; may not generalize to casual speech.
21	S. Bayerl et al. [21]	Dysfluencies Seldom Come Alone - Detection as a Multi-Label Problem	Modified wav2vec 2.0 for simultaneous multi-label detection of co-occurring dysfluencies.	SOTA on SEP-28k-Extended with cross-language generalization.	Multi-label training increases model complexity and annotation requirements.
22	R. Gong et al. [22]	AS-70: A Mandarin Stuttered Speech Dataset for Automatic Speech Recognition	First large-scale Mandarin stuttered speech dataset with verbatim transcriptions and detailed annotations.	Enables Mandarin stuttering research with 70h of diverse speaker data.	Limited to Mandarin; annotation process labor-intensive.
23	X. Liu et al. [23]	An End-to-End Stuttering Detection Method Based on Conformer and BiLSTM	Conformer blocks with BiLSTM temporal modeling for multi-task dysfluency and severity prediction.	1st place SLT 2024 Challenge; 39.8% F1 improvement on AS-70.	Conformer architecture requires significant GPU memory and training time.
24	A. Batra et al. [24]	Boli: A Dataset for Understanding Stuttering Experience	Multi-lingual Indian language dataset with read and spontaneous speech	Captures real-world Indian stuttering patterns across multiple languages.	Limited annotations; may need expert review for clinical applications.

Sl. No	Author	Title	Features	Pros	Cons
			across 5 stutter types.		
25	A. R. Valente et al. [25]	Clinical Annotations for Automatic Stuttering Severity Assessment	Enhanced FluencyBank with expert clinician annotations including audiovisual and behavioral cues.	Provides clinical-grade multi-dimensional annotations for severity assessment.	Expert annotation is expensive and time-consuming to produce.
26	T. Grósz et al. [26]	Wav2vec2-based Paralinguistic Systems to Recognise Vocalised Emotions and Stuttering	Joint wav2vec2 framework for emotion recognition and stuttering detection from speech embeddings.	Demonstrates emotion-disfluency interconnection with 62.1% UAR.	Joint modeling may introduce task interference; limited to paralinguistic analysis.
27	J. Tang et al. [27]	Speech Annotation Guidelines with People Who Stutter	Standardized annotation protocols developed collaboratively with PWS for consistent dysfluency labeling.	Improves annotation quality and reproducibility across stuttering research.	Guidelines adoption requires community-wide coordination and training.
28	A. Romana et al. [28]	FluencyBank Timestamped: An Updated Data Set for Disfluency Detection and Automatic Intended Speech Recognition	Updated FluencyBank with word-level timestamps and refined disfluency annotations for precise temporal alignment.	Enables word-level dysfluency localization and more accurate model evaluation.	Limited to English stuttering patterns; requires updated annotation protocols.
29	L. Nie et al. [29]	MMSD-Net: Towards Multi-modal Stuttering Detection	First multi-modal neural framework combining audio and visual signals through transformer-based cross-modal fusion.	2-17% F1 improvement over unimodal approaches; captures visual stuttering cues.	Requires video input; increased computational complexity and data collection overhead.
30	R. P. Buzzeti et al. [30]	Detecting Stuttering with Artificial Intelligence: A Hybrid Method for Brazilian Portuguese	Two-stage hybrid approach combining rule-based detection with ML classification for Portuguese stuttering.	Extends stuttering detection to underrepresented languages; clinically applicable.	Language-specific rules may not transfer to other languages without modification.

Table 1.1: Summary of Literature Survey

1.4 PROBLEM STATEMENT

Stuttering assessment in clinical practice still depends heavily on manual observation, where a speech-language pathologist listens to recordings and marks dysfluencies by hand. This process is slow, subjective, and difficult to reproduce, and the disagreement between raters makes it hard to track a patient's progress reliably. Existing automated tools mostly stop at binary detection, reporting that a recording contains stuttering without indicating which type occurred or where in the audio it occurred, which limits their usefulness for therapy planning.

The problem addressed in this project is therefore to build a single pipeline that takes an unconstrained speech recording and produces a structured analysis: whether stuttering is present, which of the five dysfluency types are involved, and the exact time regions where each event occurs. The system must also remain usable without large annotated corpora from the target speaker and must run fast enough to support interactive assessment rather than offline batch processing.

1.5 OBJECTIVES

The project is organized around four objectives, each contributing one stage of the final pipeline.

1. **Speech preprocessing:** clean and standardize the input audio so that every downstream model receives consistent data. This covers resampling to a fixed rate, DC offset removal, peak normalization, silence trimming, fixed-length padding, and augmentation such as noise injection and time or pitch shifting, which together reduce the effect of varying recording conditions and speaker loudness.
2. **Stutter detection:** decide whether a given speech sample contains dysfluent speech at all. The output is a binary signal that gates the rest of the analysis, so the model is trained to hold up across speakers, languages, and background noise, not only on clean studio recordings.
3. **Stutter classification:** identify which dysfluency types are present among prolongation, block, sound repetition, word repetition, and interjection. Because these events often co-occur, the system decides each type independently and gives it its own confidence score rather than forcing one mutually exclusive label.
4. **Stutter localization:** find the time regions in the waveform where the detected events occur and align them with the corresponding words in the transcript. That gives clini-

cians usable timestamps, so they can go straight to the affected segment and compare the detected events with what was said.

The objectives are ordered so each stage builds on the one before it. Preprocessing produces a normalized waveform and spectrogram for the detection and classification models. Detection then narrows the analysis to samples that contain dysfluencies. Classification assigns type-level scores to those samples, and localization maps the detected events back onto the time axis and the transcript.

CHAPTER 2

REQUIREMENT SPECIFICATION AND ANALYSIS

2.1 INTRODUCTION

The development of a deep learning-based framework for stuttering classification and localization requires a clear understanding of both clinical and technical aspects. Before implementation, the project needs to define its functional and non-functional requirements, system components, and workflow, so the result is accurate and clinically reliable. The goal is to turn manual speech-language assessment into an automated model that detects dysfluencies early and objectively. The system should process audio recordings, extract features, and classify them into distinct stuttering types. It should also locate dysfluency events within the signal. It needs to reduce the time manual analysis takes and the variation between observers, while keeping the analysis automated, precise, and easy to interpret. The framework uses Wav2Vec 2.0 for feature extraction, Wav2Vec2 for frame-level localization, and Whisper for transcription. It should also run efficiently on standard hardware, handle larger datasets as they arrive, and give clinicians and end users a simple interface. The system needs to keep data secure and confidential to meet ethical and legal standards.

2.2 FUNCTIONAL REQUIREMENTS

The functional requirements list what the system must do to detect and classify stuttering events with deep learning models. Clinicians, speech-language pathologists, and people who stutter can upload a recording or capture audio through the interface. Supported formats are WAV, MP3, FLAC, and M4A.

The system processes the audio and sorts it into five dysfluency types: prolongation, block, sound repetition, word repetition, and interjection. For each detected class, the interface shows the result and a confidence score, so users can see how reliable the prediction is. The system also locates dysfluency events and returns their timestamps. Waveforms, spectrograms, and overlays mark the regions that influenced the model's decision, which makes the output easier to interpret. Users can download the full analysis report for their records, and the system stores uploaded audio and results securely for later reference.

2.2.1 User Requirements

User requirements specify the functionalities and services that the end users should be able to access and utilize. These typically reflect how users will interact with the system. The proposed system is primarily designed for clinicians, speech-language pathologists, researchers, and individuals who stutter involved in the assessment and monitoring of speech fluency. The users of this system require an intuitive, reliable, and efficient platform that can process speech recordings and provide accurate diagnostic insights with minimal manual effort. The users expect the interface to be simple, interactive, and easy to navigate, allowing them to view classification results, visual interpretations, and confidence levels without technical complexity.

For a speech analysis platform, general user requirements include:

- The user should be able to upload or record a speech sample for analysis.
- The user should be able to access real-time classification results.
- The system should provide detailed analysis reports.

2.2.2 System Requirements

System requirements define the essential capabilities of software and hardware needed to support system functionality. The proposed stuttering detection system requires components that can efficiently handle the computational demands of deep learning-based speech analysis while delivering the core functional operations described above. Hardware guidelines appear in the hardware requirements section later in this chapter.

These requirements include:

- The system must accept audio uploads securely and classify them.
- Users need a way to save previous analysis results and open them again.
- Adequate storage must be available for managing large speech datasets.

2.3 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements set the quality, performance, and usability standards the system has to meet so that it runs efficiently and reliably. The key requirement is performance efficiency, as the models handle deep learning and audio processing tasks that must deliver fast and accurate results with minimal delay. GPU acceleration and optimized memory management help achieve smooth processing of speech recordings. Reliability is equally important: the system should maintain consistent accuracy across datasets and handle interruptions through fault tolerance. It must also be scalable,

allowing expansion for larger datasets or additional dysfluency types without compromising performance. In terms of usability, the interface should be simple, intuitive, and accessible to both technical and non-technical users for easy audio upload, analysis, and visualization. Maintainability should be ensured through modular, well-structured code that supports easy updates and retraining. Finally, portability and availability ensure the system runs smoothly across different platforms with high uptime and regular backups for uninterrupted access.

2.3.1 Performance

- The system should process and analyse a speech sample within 3-5 seconds.
- The classification results should keep false negatives and false positives low.
- Performance metrics should be monitored regularly.
- The system should handle processing of audio uploads without significant delay.

2.3.2 Scalability

- The system should efficiently handle increasing speech datasets.
- It should be scalable for integration across multiple clinics and research centres.
- The system should be adaptable for improvements in deep learning models.
- The system should be capable of handling a higher volume of concurrent users.

2.3.3 Availability

- The system should remain accessible to users at all times, ensuring minimal downtime during clinical use.
- System components are designed to be fault-tolerant to reduce the impact of failures.
- The platform should support uninterrupted service even during updates or model improvements.

2.3.4 Usability

- The user interface should be user-friendly for clinicians, speech-language pathologists, and non-technical users.
- The system should provide clear navigation for audio uploads and results.
- The visualization outputs shall help users easily understand the model's predictions for the analysed speech.

2.4 Software Requirements

The software requirements cover what is needed to develop, train, validate, and deploy the deep learning models that detect stuttering events in speech audio. A range of tools and libraries is needed to develop models, process audio, and evaluate performance.

- **Operating System:** the proposed system runs on Linux and Windows. It is fully compatible with Windows 10 and above and common Linux distributions, offering stability, wide software support, and seamless integration with deep learning frameworks and GPU drivers. This environment ensures smooth performance, easy setup, and reliable execution of the proposed models, with the PySide6 desktop application and the web application able to run on either platform and training and inference supported on CPU or NVIDIA GPU.
- **Deep Learning Framework:** the proposed system is built using PyTorch as the primary deep learning framework for building and training the machine learning models. PyTorch provides the required computational efficiency, GPU support through features such as torch.compile, mixed precision, and TensorFloat-32 (TF32), and scalability for handling complex audio classification tasks effectively. The Hugging Face Transformers library supplies the pretrained speech models, Wav2Vec 2.0 for feature extraction and Whisper for timestamped transcription.
- **Libraries:** a range of Python libraries is required to support different stages of the project:
    - librosa: Used for audio preprocessing and manipulation, including loading, resampling, mel-spectrogram computation, and silence trimming.
    - NumPy: Provides support for numerical operations and array handling.
    - Pandas: Essential for managing and analysing structured data such as labels or metadata.
    - Scikit-learn: Used for data splitting, model evaluation metrics, and traditional machine learning algorithms.

2.5 Hardware Requirements

The hardware requirements outline the minimum and recommended specifications needed to efficiently run the deep learning models used in this project. Proper hardware ensures

smooth data processing, model training, and inference performance, which is critical when working with speech audio and deep learning pipelines.

- **Processor (CPU):** a multi-core processor such as Intel Core i5 (8th generation or newer) or AMD Ryzen 5 and above is required. These processors provide adequate computational power for audio preprocessing and managing training workflows.
- **Storage (Hard Disk):** at least 10 GB of free disk space is required to store datasets, trained model checkpoints, logs, and temporary files. SSDs are recommended for faster read/write speeds and improved data handling.
- **Memory (RAM):** a minimum of 8 GB RAM is necessary to run the training scripts and handle datasets. However, 16 GB or more is recommended for better performance, especially during the training and validation phases when data loading and augmentation are involved.
- **Graphics Processing Unit (GPU):** for faster model training and efficient computation, an NVIDIA GPU with CUDA support is highly recommended. GPUs such as the NVIDIA RTX 2060 or higher (e.g., RTX 3060, RTX 3080) can greatly reduce training time and handle larger datasets and complex neural network architectures effectively. Proper installation of CUDA and cuDNN is required to ensure full compatibility with PyTorch.
- **Audio Input Device:** a functional microphone is required for recording speech input within the application.

The minimum configuration is sufficient for inference on short clips, while training or fine-tuning the larger pretrained backbones benefits from the recommended configuration. A GPU with at least 8 GB of memory is preferable when fine-tuning, although the classification-only mode runs comfortably on a CPU.

CHAPTER 3

SYSTEM DESIGN

3.1 INTRODUCTION

System design is an essential stage in software development that focuses on defining the overall structure and functionality of a system. It translates user and technical requirements into a detailed framework that outlines how different components, modules, and interfaces will interact. The process involves creating both high-level architecture, which defines the system's major elements and their interconnections, and low-level design, which specifies detailed components such as data structures, algorithms, and communication flows.

The main objective of system design is to build a solution that fulfills user needs while ensuring performance, reliability, scalability, and security. It also emphasizes maintainability and flexibility so the system can adapt to future enhancements. Effective system design requires collaboration among developers, architects, and domain experts to ensure the architecture aligns with functional goals and business requirements. By carefully organizing how each part of the system operates and communicates, system design establishes a strong foundation for efficient implementation and long-term stability.

3.2 SYSTEM ARCHITECTURE AND DESIGN

The system architecture defines the overall structure and organization of the Stuttering Classification and Localization System using deep learning. It provides a comprehensive framework describing how different components interact to process speech audio, extract discriminative features, and classify utterances into normal and dysfluent categories. A well-structured architecture ensures efficient data flow, modularity, and scalability, allowing the model to deliver high diagnostic accuracy while remaining adaptable to future advancements. The architecture covers the main stages of the pipeline, from preprocessing and feature extraction to model training and classification, so that performance remains stable. Furthermore, it supports seamless integration with user interfaces for audio upload, visualization, and prediction display. Emphasis is placed on maintaining data security and patient confidentiality, ensuring ethical and reliable operation throughout the analysis workflow. Additionally, the architecture allows for performance optimization, enabling fast and accurate stutter detection even when deployed on large speech datasets.

The proposed deep learning architecture for the automatic classification and localization of stuttering using speech audio is illustrated in Figure 3.1. The framework integrates two model pathways, five independent Wav2Vec 2.0 binary classifiers for classification and a Wav2Vec2 frame-level pipeline for localization, to improve feature extraction, multi-perspective learning, and classification robustness. The complete workflow consists of five major phases: Input, Pre-processing, Feature Extraction, Classification, and Output. This dual-model structure enables richer representation learning and enhances overall detection accuracy.

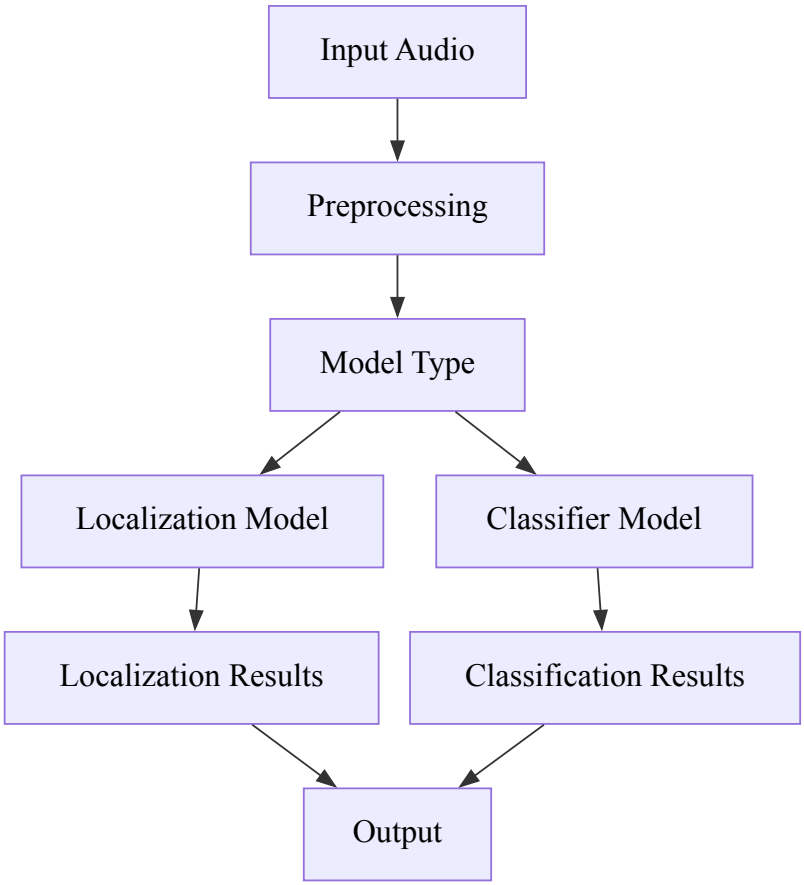


Figure 3.1: System Architecture

Speech Audio Input: The system begins with the collection of speech audio samples, which serve as the primary dataset for stutter detection. Audio enters the system when the user records directly or uploads a pre-recorded file through the web interface or desktop application. The web app captures audio with MediaRecorder APIs; the desktop app uses sounddevice. Each sample may contain one or more dysfluencies, including blocks, prolongations, word repetitions, sound repetitions, and interjections. Ensuring adequate recording quality is crucial, as variations in microphone hardware, background noise, and speaking rate can affect model reliability. The dataset includes recordings from Project Boli, SEP-28K, and UCLASS, and the ingestion pipeline resolves the English material of

all three sources. This stage gives the diagnostic system its high-quality data for the steps that follow.

Preprocessing: Before being fed into the deep learning model, speech audio undergoes preprocessing to enhance clarity, reduce noise, and ensure consistency across samples. The audio is sent to the backend and converted to 16 kHz mono WAV with FFmpeg so every clip is handled the same way. The preprocessing pipeline also includes DC offset removal, peak normalization, and silence trimming to clean the signal and remove unwanted artifacts. Fixed-length padding truncates or pads each clip to 48,000 samples, three seconds at 16 kHz, so all inputs have the same length. After this step the input audio is standardized and optimized, so the deep learning architecture can extract features and classify it accurately.

Wav2Vec 2.0: The detection framework starts with Wav2Vec 2.0, a deep self-supervised model pre-trained on large amounts of unlabeled speech. Wav2Vec 2.0 quantizes the raw waveform into contextualized speech representations, learning robust acoustic features without requiring manually labeled data. This structure helps the model retain important information and capture fine-grained phonetic and prosodic patterns, such as broken phonation, repetitions, and elongated sounds, which are key indicators of dysfluent speech. Because it is pre-trained on large speech corpora, the model already understands general speech features, so adapting it to stutter detection with limited labeled data is easier. Because it extracts contextual features at several levels, it gives a strong base for detecting stutter events with high precision.

Detection and Localization Pipelines: To achieve higher accuracy, the proposed system uses a hybrid detection framework that combines classification and localization. After preprocessing, the audio runs through three parallel pipelines: classification, transcription, and localization. The classification pipeline extracts speech representations with Wav2Vec 2.0 and passes them through five independent binary classifiers, each trained for a distinct dysfluency type. The localization pipeline identifies dysfluency regions at the frame level using Wav2Vec2 frame-level feature extraction, so it can mark the precise segments of speech where dysfluencies occur. Combining the two approaches lets the system catch fine acoustic detail and broader prosodic structure, so results stay stable when recordings vary in noise, clarity, or speaking style.

Classification: Once feature extraction finishes, the Wav2Vec 2.0 representations go to the classification stage. Here, five independent binary classifiers assign a presence probability to each dysfluency type, using decision boundaries learned during training to separate fluent from dysfluent speech. Sigmoid outputs give per-class probabilities,

and binary cross-entropy loss drives optimization during training. Their outputs are aggregated into a multi-label result that reports the presence probability of each dysfluency type and summarizes the detected classes and the primary dysfluency. A multitask shared-backbone variant was also trained for comparison, and its outputs are combined with localization at prediction time: the combiner labels each localized region with per-class saliency scores and fuses the two views into a single annotated result. The output gives speech-language pathologists diagnostic results they can act on when identifying stutter events.

Transcription and Alignment: The transcription pipeline uses the Whisper Automatic Speech Recognition (ASR) model to produce a timestamped transcript, with support for English, Kannada, and Hindi. A Connectionist Temporal Classification (CTC) based time alignment step ties the pipelines together, mapping the detected dysfluency regions to specific words or syllables in the transcript. Language-specific adapters keep the alignment accurate for English, Kannada, and Hindi.

Output: In the final stage, the system presents its predictions through a user-friendly interface. The output typically shows the detected dysfluency types along with confidence scores that indicate how certain the model is about each decision, and a summary of the primary dysfluency. To make the results more interpretable, the outputs surface as waveform overlays, spectrogram visualizations, timestamped transcripts, and a detailed clinical-style report. The outputs are designed to be simple, clear, and clinically relevant, making them suitable for practical use in clinical or self-assessment settings. By providing accurate and interpretable results, the system supports early assessment and can play a valuable role in improving outcomes for people who stutter.

Both the web and desktop applications load trained models through the centralized model registry. The registry reads from a registry module (the model/registry package) and a configuration file (registry.json), so checkpoints can be updated or replaced without changing application code, which keeps the system flexible and easy to maintain.

3.3 FLOWCHART

The process begins with the input audio, recorded or uploaded by the user. The audio is converted to 16 kHz mono WAV with FFmpeg, then padded or truncated to 48,000 samples, three seconds at 16 kHz, so all inputs have the same length.

After normalization, the audio runs through three parallel pipelines: classification, transcription, and localization. Classification produces dysfluency probabilities, transcription a timestamped transcript, and localization frame-level dysfluency regions.

A CTC-based alignment step combines these outputs and maps the detected dysfluencies to specific words or syllables. The final data is turned into per-word annotations, shown in waveform and spectrogram displays, and written into the analysis report.

The system also defines a structured data flow for training. Training uses three datasets: Project Boli (sourced via Git repositories), SEP-28K (approximately 28,000 audio clips), and UCLASS (both obtained via Kaggle). Each dataset goes through its own normalization functions.

The processed datasets merge into a unified format: a combined_labels.csv file with multi-label binary annotations, plus individual interval CSV files for each audio clip. The merged dataset is split into training, validation, and testing subsets with an 80:10:10 split, with the training subset further re-split 80/20 during model training. The subsets are organized with symbolic links for efficient access, and preprocessed audio files are cached to speed up training.

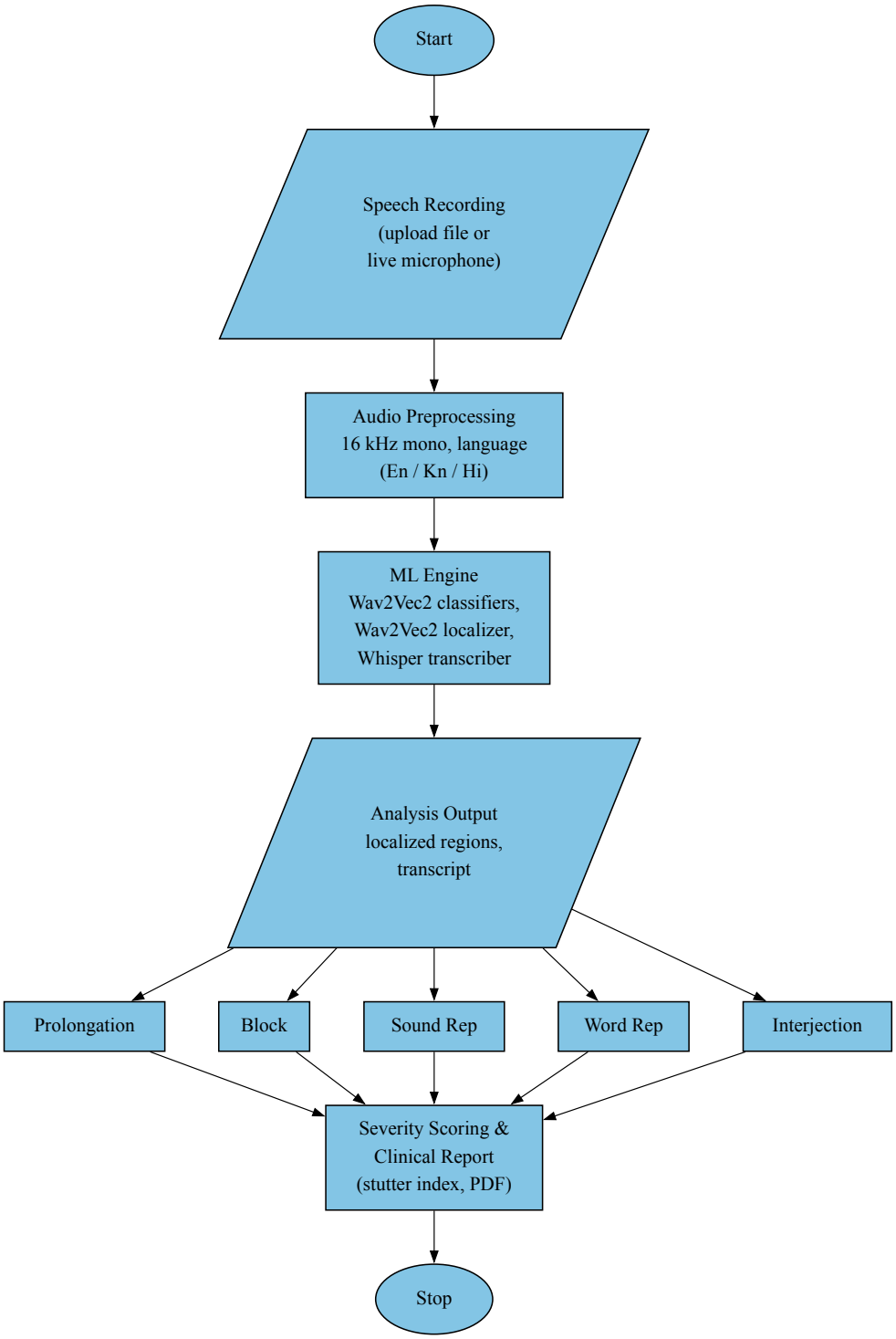


Figure 3.2: Flowchart

The flowchart shown in Figure 3.2 illustrates the workflow for stutter detection using the proposed hybrid deep learning approach. The process begins with the collection of speech audio recordings, followed by preprocessing steps such as conversion to 16 kHz mono WAV, DC offset removal, peak normalization, silence trimming, and fixed-length padding to improve signal quality and variability. The refined audio is then split into training, validation, and testing datasets with an 80:10:10 split. Initially, the Wav2Vec 2.0 model is used to extract high-level contextual speech features. To enhance performance, a hybrid model combining five independent Wav2Vec 2.0 binary classifiers with a

Wav2Vec2 frame-level localizer is developed, where the classifiers capture dysfluency-specific acoustic patterns and the localizer identifies fine-grained temporal segments. The outputs from both pipelines are fused through CTC-based alignment and passed through a final aggregation layer that categorizes the audio samples as containing blocks, prolongations, sound repetitions, word repetitions, or interjections, along with the precise localized regions. This flowchart provides a clear overview of the systematic steps involved in the proposed framework for stutter detection.

3.4 USE CASE DIAGRAM AND DESCRIPTIONS

The use case diagram shows how the user interacts with the system. The primary actor is the User, either a clinician or an individual using the application for self-assessment.

The main workflow covers recording audio, uploading pre-recorded audio, and running speech analysis in either full mode or classification-only mode. After analysis, users can view waveform visualizations, spectrograms, transcripts, and stutter detection results.

Users can also localize dysfluencies within the speech, generate analysis or clinical reports, and revisit previously recorded sessions through a history feature. Supporting actions include toggling between interface themes and accessing standardized reading passages for consistent evaluation. The diagram keeps the user in a single role so that clinicians and individuals follow the same path, from capturing speech to reviewing the outcome. Results are stored per session, which lets a user compare recordings over time and monitor progress across assessments.

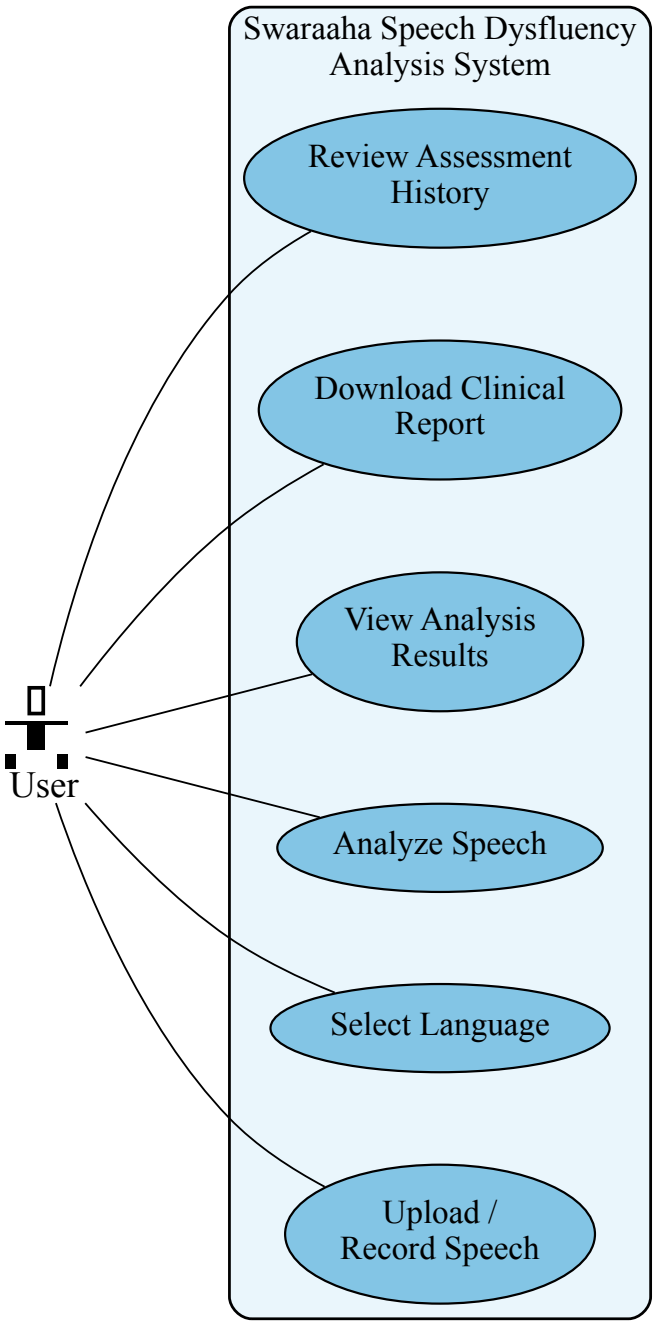


Figure 3.3: Use Case Diagram

The use case diagram shown in Figure 3.3 illustrates the interactions between the user and the stutter detection system. The user can register or log in, then either record audio through the microphone or upload a pre-recorded speech file, which is analyzed through the full or classification-only pipeline. The system displays the classification results with confidence scores, localized dysfluency regions, and a timestamped transcript, and allows the user to generate and download analysis or clinical reports. Additional use cases include viewing the session history, switching between light and dark themes, and accessing standardized reading passages for consistent assessment. This diagram provides a clear overview of the functional interactions supported by the proposed framework.

3.5 WORKFLOW DIAGRAM

A workflow diagram is a structured visual representation of a sequence of operations or tasks carried out to complete a specific process, typically used to analyse, design, or manage complex systems. It employs standardized symbols such as rectangles to denote actions, diamonds for decision points, and arrows to indicate the direction of flow, thereby offering clarity and insight into the procedural steps involved. In the context of deep learning and stutter detection, the workflow diagram illustrated here systematically maps out the process for developing a stutter classification and localization system.

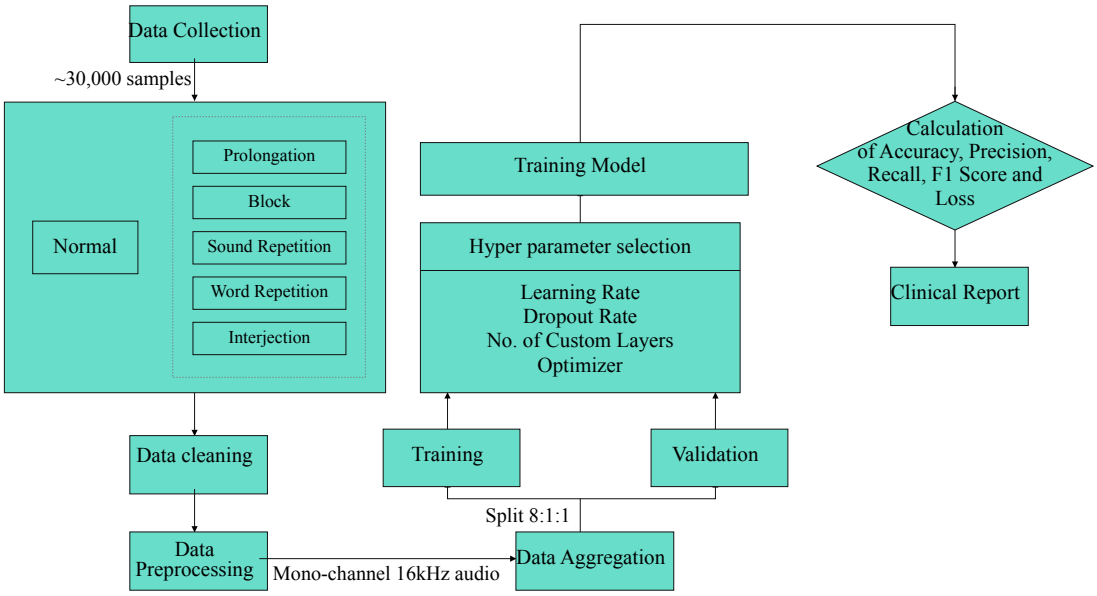


Figure 3.4: Workflow Diagram

The workflow of the proposed deep learning pipeline for stutter detection and localization is shown in Figure 3.4. The process begins with the collection of speech audio samples from three public stuttering datasets, Project Boli, SEP-28K, and UCLASS, all converted to 16 kHz mono and standardized through DC offset removal, peak normalization, silence trimming, and fixed-length padding. Data augmentation techniques such as random noise injection, time stretching, pitch shifting, temporal shifting, and amplitude scaling are applied to the waveforms, along with time and frequency masking on the spectrograms, expanding the effective dataset so the models generalize well across varied speaking patterns. The processed audio is then split into training, validation, and testing sets using an 80:10:10 ratio, with the training subset further re-split 80/20 during training. Model development begins with fine-tuning a pre-trained Wav2Vec 2.0 backbone, followed by a hybrid architecture that combines five independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer to capture both global dysfluency patterns and fine-grained temporal segments. Hyperparameters including learning rate, backbone freeze

duration, optimizer, and the number of trainable layers are tuned to enhance performance, and the trained network is periodically validated to ensure stable learning and prevent overfitting. Finally, evaluation metrics such as precision, recall, F1-score, AUROC, and mean IoU are computed, and the best-performing model is selected for reliable automated assessment of speech dysfluencies.

3.6 SEQUENCE DIAGRAM

The sequence diagram shows how the components interact during execution. The user works through the graphical user interface (GUI), which sends a request to the backend API endpoint (/api/analyze).

The backend runs the request through a chain of services: preprocessing, classification, transcription, localization, and alignment. Each service does its part and passes its output to the next stage. When processing completes, the results go back to the frontend, where the user sees them. The system also stores the results for report generation and history management. The sequence diagram shown in Figure 3.5 illustrates the interaction flow between the user, the frontend, the backend services, and the stored results during a speech analysis session. The user initiates the analysis through the graphical user interface, which sends a request to the backend API endpoint. The backend sequentially invokes the preprocessing, classification, transcription, localization, and alignment services, each consuming the output of the previous stage. Once the processing completes, the fused results are returned to the frontend for display and saved in the local storage for reports and the session history. This diagram provides a clear overview of the message exchange that drives the proposed framework.

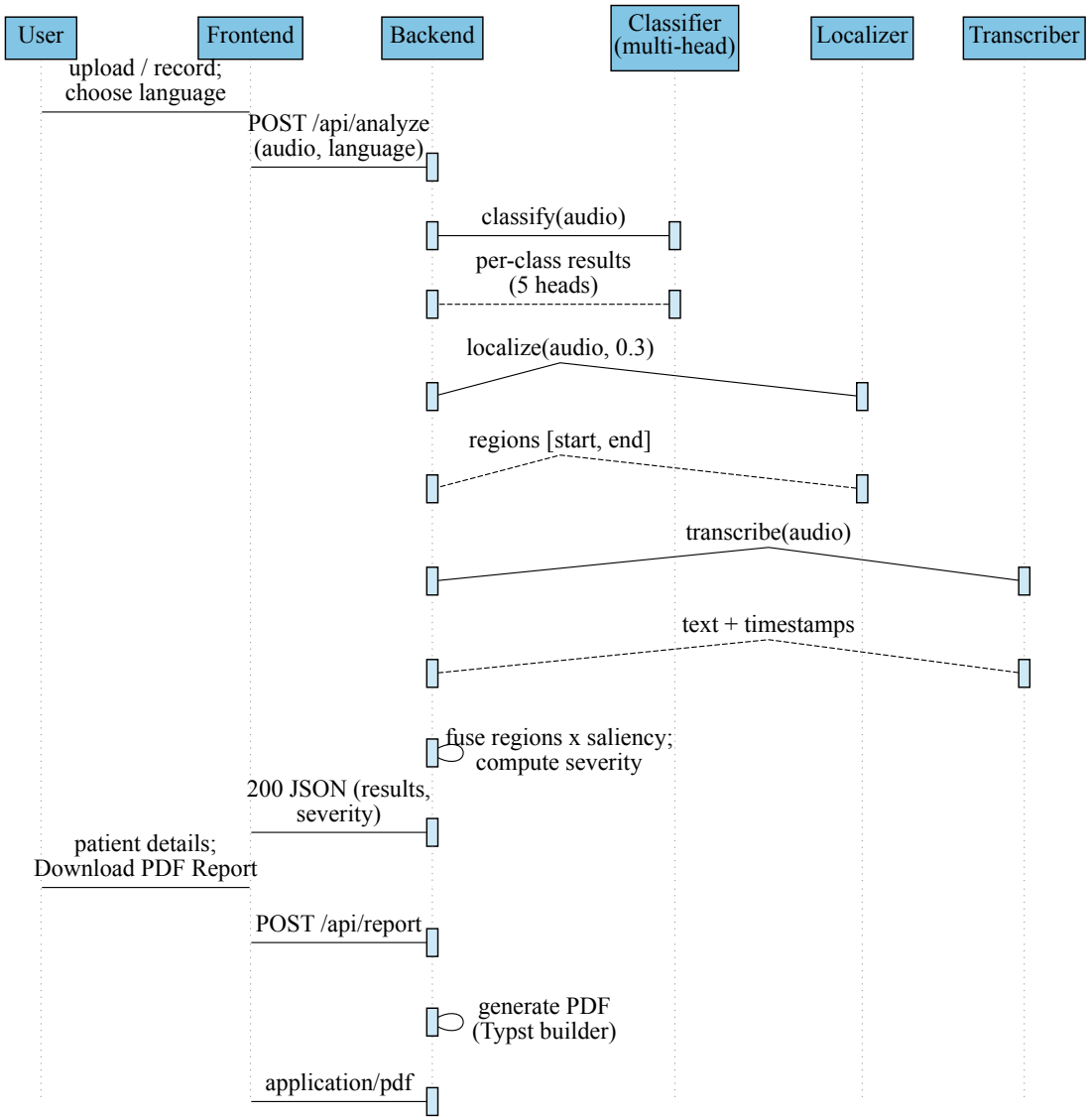


Figure 3.5: Sequence Diagram

The sequence diagram (Figure 3.5) captures the end-to-end interaction model: the GUI acts as the entry point, the backend coordinates the analysis pipeline, and the results layer handles persistence and presentation. This separation of concerns ensures modularity, traceability of requests, and consistent formatting of outputs across the web and desktop interfaces. Together, these interactions ensure that each analysis request is handled reliably from input to report generation, with clear responsibility boundaries between components and minimal coupling between the frontend and backend services.

The request lifecycle begins when the user confirms an analysis, at which point the frontend packages the audio, the selected mode, and any optional parameters into a single request. The backend validates the payload before any model is invoked, rejecting unsupported formats or empty inputs early so that expensive inference is never wasted on malformed data. Preprocessing normalizes the waveform and produces the log-mel spectrograms and frame-level features that the downstream stages expect, acting as the

single point where audio conventions are enforced. Classification then assigns probability scores to each dysfluency type, and transcription produces a timestamped transcript that the alignment stage later couples to the detected events. Localization runs in parallel with transcription where possible, since the two stages share the same preprocessed audio and do not depend on each other's output, which keeps the overall latency low. The alignment stage fuses word timings with the localized regions so that each dysfluency can be reported in the context of the spoken words around it. If any single service fails, the backend returns a structured error instead of a partial result, and the frontend surfaces the failure without discarding the user's audio, allowing the request to be retried. On success, the fused payload is stored locally so that reports and the session history can be reconstructed later without re-running the models, which also keeps repeated report generation fast and independent of the analysis services. This clear division between request handling, computation, and persistence is what allows the same backend to serve both the web and desktop interfaces while keeping behaviour identical across them.

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 INTRODUCTION

The system implementation process is a multifaceted endeavor focused on developing a robust software architecture capable of effectively managing diverse user interactions, data processing tasks, and model inference operations. By employing meticulous design and coding practices, the implementation ensures that the application is scalable, reliable, and secure, capable of handling varying workloads and protecting sensitive data. Iterative testing and debugging procedures are integral components of the implementation process, allowing for the identification and resolution of any issues or bugs that may arise. This iterative approach ensures that the application delivers a seamless user experience, free from disruptions or errors. Additionally, post-deployment monitoring and maintenance mechanisms are established to continually assess system performance and address evolving requirements. This ongoing maintenance ensures that the system remains optimized and responsive to user needs, even as conditions change over time, thereby facilitating long-term success and user satisfaction.

4.2 ALGORITHM

The inference process follows a fixed sequence of steps that turns raw speech input into a complete, annotated analysis result.

- Step 1, Input Acquisition: the system acquires speech input either through real-time recording or by uploading an audio file.
- Step 2, Audio Conversion: the input audio is converted into a 16 kHz mono WAV format using FFmpeg to maintain consistency across all inputs.
- Step 3, Preprocessing: the audio is cleaned by removing the DC offset, applying peak normalization to a target amplitude, and trimming silent segments.
- Step 4, Length Normalization: the processed audio is adjusted to a fixed length of 48,000 samples (three seconds at 16 kHz) by padding or truncating as required.
- Step 5, Parallel Processing: the system processes the audio simultaneously through three independent branches:

1. Classification: Wav2Vec 2.0 embeddings are generated and passed through five binary classifiers, one per dysfluency type, and the outputs are aggregated into a multi-label result with a probability score for each dysfluency type.
2. Transcription: the Whisper model generates a timestamped transcript of the speech in English, Kannada, or Hindi.
3. Localization: the Wav2Vec2 frame-level localizer identifies the frame-level regions within the audio where dysfluencies occur.

- Step 6, Alignment: the detected dysfluency regions are mapped to the corresponding words or syllables in the transcript using CTC-based time alignment.
- Step 7, Combination and Severity: the combiner labels each localized region with per-class saliency scores from the classifier and fuses the classification and localization views, and the severity module computes a stutter index from the ratio of dysfluent speech duration to total speech duration.
- Step 8, Visualization and Report Generation: the system displays the results through waveform overlays, spectrograms, transcripts, and confidence scores, and generates a detailed clinical-style report.

4.3 IMPLEMENTATION REQUIREMENTS

This section presents the implementation requirements for detecting and localizing stuttering with a hybrid deep learning model that combines Wav2Vec 2.0 feature extraction, five independent Wav2Vec 2.0 binary classifiers, a Wav2Vec2 frame-level localizer, and Whisper automatic speech recognition. The main objective is to build a reliable, efficient system that identifies speech dysfluencies and localizes them within the audio. The hybrid model uses the strengths of both architectures: Wav2Vec 2.0 supplies self-supervised, contextual speech representations that capture fine-grained phonetic and prosodic patterns, while Whisper provides sequence-level transcription that helps map localized dysfluency regions to specific words and syllables.

The implementation uses the Python ecosystem, which provides an interactive environment for model development, experimentation, and analysis. Python serves as the primary language for backend development because of its versatility and its support for a wide range of scientific and deep learning libraries such as PyTorch, the Hugging Face Transformers library, and NumPy, which help with model construction, training, and evaluation. For the frontend, React 19 and PySide6 build a user-friendly web interface and desktop application where users can record or upload speech audio and view classi-

fication and localization results. All trained models are managed by a centralized model registry configured through a registry.json file that maps each task to its checkpoint and per-class thresholds, so checkpoints can be updated or replaced without changing application code. The Python ecosystem also facilitates matrix manipulation, data visualization, algorithm implementation, and seamless integration with other programming environments, making it an ideal platform for developing and testing the hybrid model for stuttering classification and localization.

4.3.1 Train the Model

Training is a key step in developing the system for stutter classification and localization. The speech audio dataset is first collected from Project Boli, SEP-28K, and UCLASS, then preprocessed through conversion to 16 kHz mono WAV, DC offset removal, peak normalization, silence trimming, and fixed-length padding, with data augmentation to improve audio quality and reduce overfitting. The Wav2Vec 2.0 model is then trained to extract high-level contextual speech features, using its self-supervised pretraining to learn robust representations. To improve classification accuracy further, the hybrid model combines five independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer. The localizer captures fine-grained temporal features at the frame level, complementing the broader segment-level extraction of the classifiers. CTC-based time alignment then fuses the outputs of both models into one feature representation. Training uses the AdamW optimizer to adjust learning rates adaptively and keep convergence stable. It also uses mini-batches of audio, and backpropagation updates the weights from the computed loss. Validation runs periodically to monitor accuracy and prevent overfitting. After training, the optimized parameters and weights are saved, so the models can predict on new speech audio without retraining.

4.3.2 Test the Model

Testing happens after training to evaluate how well the proposed model detects stutter events in unseen speech audio. When a user records or uploads a sample through the web interface or desktop application, it goes through the same preprocessing used in training: conversion to 16 kHz mono WAV, DC offset removal, peak normalization, silence trimming, and fixed-length padding. This keeps the input uniform for accurate analysis. The trained Wav2Vec 2.0 model then extracts high-level contextual speech features. For higher accuracy, the audio also runs through the hybrid model that combines five independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer, which captures both segment-level dysfluency patterns and fine temporal detail. CTC-based time alignment fuses the features from both models into one representation. The final

aggregation layer then produces a classification result that names the dysfluency types present (blocks, prolongations, sound repetitions, word repetitions, or interjections) and the precise regions involved. The interface shows the outcome with waveform overlays, spectrograms, a timestamped transcript, and confidence scores. The model's predictions are also compared with ground truth labels to measure accuracy and generalization. This phase confirms that the hybrid model performs well on new and unseen samples, which points to reliable stutter classification and localization in real clinical use.

4.3.3 Model Evaluation

Model evaluation is essential for assessing the overall performance and reliability of the proposed system. Once the models are trained and tested, several quantitative metrics measure their classification performance:

- Accuracy: measures the proportion of correctly predicted samples.
- Precision: the share of predicted positive instances that are actually positive.
- Recall (Sensitivity): measures how effectively the model identifies all relevant positive samples.
- F1-Score: provides a harmonic mean of precision and recall to balance both metrics.

The five Wav2Vec 2.0 binary classifiers, one per dysfluency type, are further measured with AUROC (Area Under the Receiver Operating Characteristic Curve), AUPRC (Area Under the Precision-Recall Curve), and specificity, summarized per class and as a macro average across the five dysfluency classes. The Wav2Vec2 frame-level localizer is measured with frame-level precision, recall, and F1, detection accuracy, mean Intersection over Union (IoU) between predicted and ground-truth regions, and the false alarm rate per minute.

A confusion matrix for each of the five binary classifiers shows the distribution of predictions, including true positives, false positives, true negatives, and false negatives. This evaluation shows the strengths and weaknesses of each class; interjections, for example, are detected most reliably, while blocks are the hardest.

All reported results use the held-out test set of 3,715 clips, which is never touched during training or threshold selection, together with a cross-corpus Boli set that checks how well the models transfer to unseen speakers.

4.4 USER INTERFACE

The web-based interface connects the user to the proposed stutter detection and localization system. Users record speech or upload audio files for analysis. Once audio is provided, the backend system carries out the preprocessing steps, followed by feature extraction, classification, transcription, and localization using the trained models. Initially, the Wav2Vec 2.0 model is used to extract high-level contextual speech features, and to achieve higher accuracy, the hybrid pipeline combining five independent Wav2Vec 2.0 binary classifiers with the Wav2Vec2 frame-level localizer is employed. Within a few seconds, the system presents the predicted result on the screen for each of the five dysfluency types, along with confidence scores, the localized regions, and the timestamped transcript. The tool is designed for clinicians, speech-language pathologists, researchers, and individuals who stutter.

The system provides two interfaces to the same analysis pipeline: a web application and a desktop application. The web frontend is built with React 19, TypeScript, and Tailwind CSS, with a sidebar layout covering the input page, results page, history, and reading passages. The input page supports audio file upload (WAV, MP3, FLAC, M4A, OGG, WMA) and direct microphone recording through the MediaRecorder API, with client-side validation before the file is sent to the backend as multipart form data. The results page shows classification results with confidence scores, an interactive waveform with color-coded dysfluency region overlays, a timestamped transcript, and a button to generate and download a PDF report. The backend runs on FastAPI served through Uvicorn, exposing the /api/classify, /api/localize, /api/analyze, and /api/report endpoints. Assessment history stays in the browser: audio files go into IndexedDB for offline access and analysis metadata into localStorage.

The desktop application is built with PySide6 (Qt for Python) for a native cross-platform interface. It includes modules for audio recording and playback (sounddevice), file handling (soundfile), and model inference, calling the shared model package directly with no separate server, so it works fully offline. PDF report generation uses Typst with pypdfium2 for viewing.

Both interfaces share the same model package, preprocessing conventions, and result schema, so a recording analysed in the browser and the same recording analysed on the desktop produce identical predictions and reports. The web client keeps long-running requests responsive by streaming progress updates and disabling duplicate submissions while an analysis is in flight, while the desktop client runs inference in a worker thread so the window stays interactive during transcription and localization. Theme switching,

keyboard navigation, and consistent layout across window sizes are handled at the component level, which keeps the interface usable for clinicians who move between machines and screen sizes. Uploaded audio is validated on the client before it reaches the backend, and the results view keeps the waveform, transcript, and per-class confidence table in sync so that selecting a timestamp jumps to the corresponding part of the recording. Report generation is deliberately decoupled from analysis: the stored result payload is sufficient to rebuild the PDF, which means a report can be regenerated with a different layout or header without repeating the expensive model inference. This shared design also simplifies maintenance, because a change to the result schema or the report template propagates to both interfaces from a single source rather than being reimplemented twice.

CHAPTER 5

SYSTEM TESTING

5.1 INTRODUCTION

Testing is a crucial phase in the development of the proposed system for stutter detection and localization, ensuring the overall accuracy, stability, and reliability of both the individual models and the hybrid pipeline. Roughly one percent of people worldwide are affected by stuttering, and diagnosis today still relies mostly on manual assessment by speech-language pathologists, which is subjective, time-consuming, and not available to everyone, making early and reliable detection essential. In this research, Wav2Vec 2.0-based models were initially utilized to classify speech dysfluencies from audio. To further enhance diagnostic precision, a hybrid framework was developed by combining five independent Wav2Vec 2.0 binary classifiers with the Wav2Vec2 frame-level localizer and Whisper automatic speech recognition, enabling classification, localization, and transcription in a single pipeline. The testing process involved validating every stage of the pipeline, including data preprocessing, model training, classification, localization, and web-based visualization, through unit, integration, functional, and system tests. Comprehensive testing confirmed that the proposed system satisfies all functional and non-functional requirements while maintaining reliable classification and localization. Performance evaluation using precision, recall, F1-score, AUROC, AUPRC, specificity, and confusion matrix analysis, along with evaluation on the held-out test set and a cross-corpus set, demonstrated the robustness and generalization ability of the models. The results are broken down by model family and dysfluency class, and the two model families, the Wav2Vec2-based models and the CNN-based models, are compared against each other and analyzed for their strengths and weaknesses.

5.2 TYPES OF TESTS

Four types of tests were performed: unit testing, integration testing, functional testing, and system testing. Together they verify the correctness of individual components, the interfaces between them, the complete feature set, and the behavior of the finished system.

5.2.1 Unit Testing

Unit testing isolates the smallest blocks of code, specific functions and classes, to verify they work as expected. The project uses pytest suites organised under model/tests, backend/tests, app/tests, and shared/tests.

Unit tests verify the preprocessing steps (sample-rate conversion, DC offset removal, peak normalization, silence trimming, and fixed-length padding), dataset normalization and the merged combined_labels.csv format, model registry checkpoint loading, the combiner and CTC alignment logic, and the computation of the evaluation metrics. On the desktop side, individual widgets such as the waveform view, results panel, and audio handler are tested in isolation. Ruff enforces consistent code style across the codebase.

5.2.2 Integration Testing

Integration testing ensures that software modules, when connected, work together seamlessly. The backend tests exercise the FastAPI endpoints (/api/analyze, /api/classify, /api/localize, and /api/report), which chain preprocessing, classification, transcription, localization, and fusion into a single analysis response. The classification, fusion, localizer, severity, and report generator services are tested as combined units, and the model registry is checked against the classifier and localization pipelines. The desktop application calls the shared model package directly, and its model runner is integration-tested against the same service layer.

5.2.3 Functional Testing

Functional testing validates the complete set of features against the specifications under typical and edge cases. The workflow of recording or uploading audio, running the analysis, and viewing the results page with classification scores, waveform overlays, and the timestamped transcript is exercised end to end. Report generation and download, severity mapping, multilingual transcription for English, Kannada, and Hindi, and history storage through IndexedDB and localStorage are all verified against the requirements defined in Chapter 2.

5.2.4 System Testing

System testing subjects the whole software product to end-to-end scenarios in conditions that resemble the user environment. The React web application was exercised in supported browsers and the PySide6 desktop application in a standalone offline setup with no separate server, both against the same machine learning backend. Supported audio formats (WAV, MP3, OGG, and M4A) were validated across the upload, drag-and-drop, and microphone capture paths, corrupted files were rejected with clear error messages, and analysis turnaround was confirmed to stay within a few seconds per clip as required by the performance requirements in Chapter 2. The combined system-level suite confirmed that the application meets its functional and non-functional requirements.

5.3 RESULT AND ANALYSIS

This section presents the evaluation of the trained models: how the Wav2Vec2-based and CNN-based model families were trained and evaluated, the hyper parameter tuning applied, and a comparative performance analysis of all architectures.

5.3.1 Model Evaluation of the Wav2Vec2 Classifiers

The model registry defines two Wav2Vec2-based classification configurations: the default single configuration of five independent Wav2Vec2 binary classifiers, and a multitask variant that shares a single backbone and five classification heads. This section first analyzes the multitask variant in detail and then reports the shipped single configuration; the confusion matrices at the end of the section come from the shipped single configuration.

The multitask variant uses a single Wav2Vec2-base backbone with five shared binary heads. Training used Focal Loss (gamma=2.0), backbone freezing for 3 epochs, AdamW (LR 3×10^{-5}), a 500-step warm-up, and early stopping with a patience of 5. Results on 3,715 held-out test samples at the default threshold of 0.5 are in Table 5.1.

Class	**Precision**	**Recall**	**F1**	**AUROC**	**AUPRC**	**Support**
Prolongation	0.566	0.440	0.495	0.845	0.491	400
Block	0.584	0.264	0.364	0.768	0.444	553
Sound Repetition	0.705	0.502	0.586	0.878	0.624	494
Word Repetition	0.621	0.399	0.486	0.855	0.550	366
Interjection	0.810	0.746	0.776	0.935	0.848	834
Macro Average	–	–	**0.542**	–	–	**3,715**

Table 5.1: Multitask Wav2Vec2 classifier results

The per-class support counts cover positive samples; the macro row reports the total of 3,715 test clips because a clip may contain several stutter types at once.

The macro F1 at the default threshold is 0.542. Interjection performs best (F1=0.776, AUROC=0.935), which makes sense because filler words like “um” and “uh” have distinct acoustic patterns. Block is the hardest class (F1=0.364); silent pauses and hesitations are hard to distinguish from normal speech pauses.

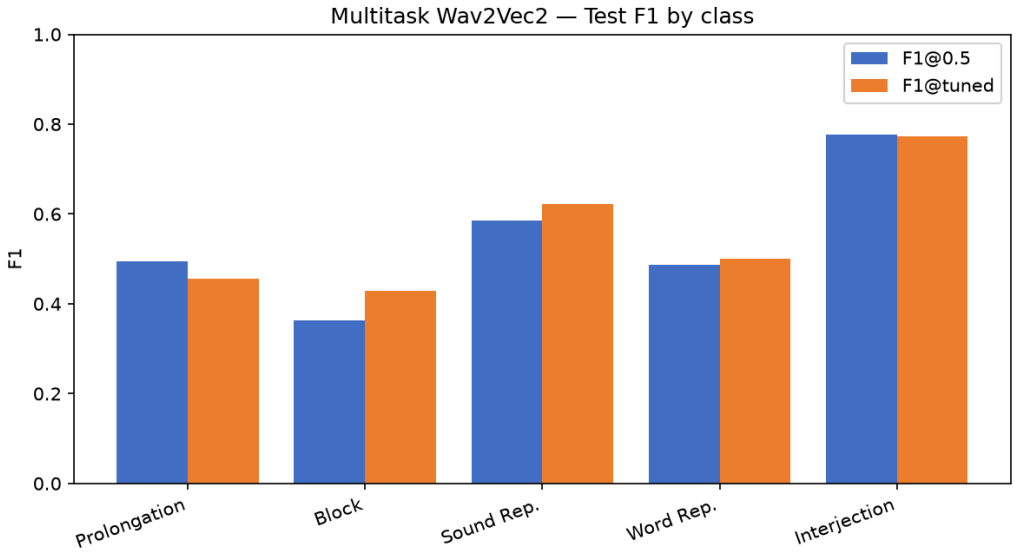


Figure 5.1: Per-class F1 of the multitask Wav2Vec2 classifier

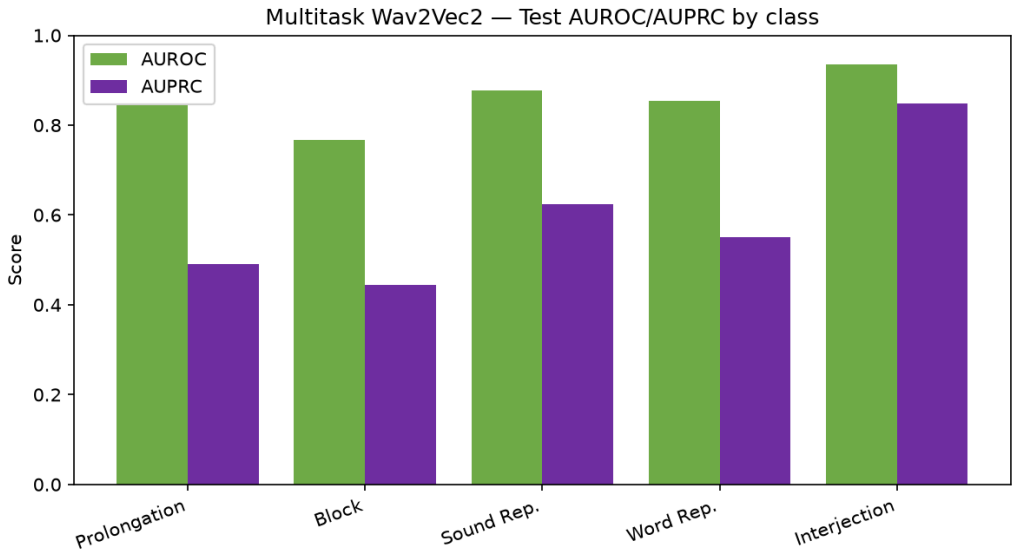


Figure 5.2: Per-class AUROC and AUPRC of the multitask Wav2Vec2 classifier

The per-class F1 scores are plotted in Figure 5.1, and the corresponding AUROC and AUPRC values are plotted in Figure 5.2. Both figures mirror the table above: interjection leads on every measure, while block trails on F1 even though its AUROC and AUPRC stay reasonably high.

The shipped single configuration of five independent binary classifiers scores slightly higher in distribution, with a test F1 of 0.573 at threshold 0.5 (0.570 with tuned thresholds).

The confusion matrices for the five dysfluency types are shown in Figure 5.3 and visualize the distribution of predictions of the shipped single configuration of five independent binary classifiers.

Before examining the matrices, it is useful to recall how the aggregate scores were obtained. Each binary classifier was trained with a class-balanced objective, and the operating threshold was selected on the validation split rather than the test split, so the matrices show the behaviour of the shipped configuration at its default threshold rather than a threshold tuned to inflate the reported numbers. Reading them row by row reveals how often each event type is correctly recovered, while reading them column by column shows which fluent or other-type segments are most likely to be mistaken for it. This distinction matters in practice, because a clinician cares both about missed events and about false alarms that interrupt a session.

The matrices are arranged so that the true label is on one axis and the predicted label on the other, with darker cells indicating larger counts. A classifier that simply predicted the majority class would produce a single dark row, which none of the five matrices does, confirming that each head learned a genuine decision boundary rather than collapsing to the dominant label. The remaining off-diagonal mass is concentrated in specific cells, which points to concrete confusions that later sections address through threshold tuning and post-processing.

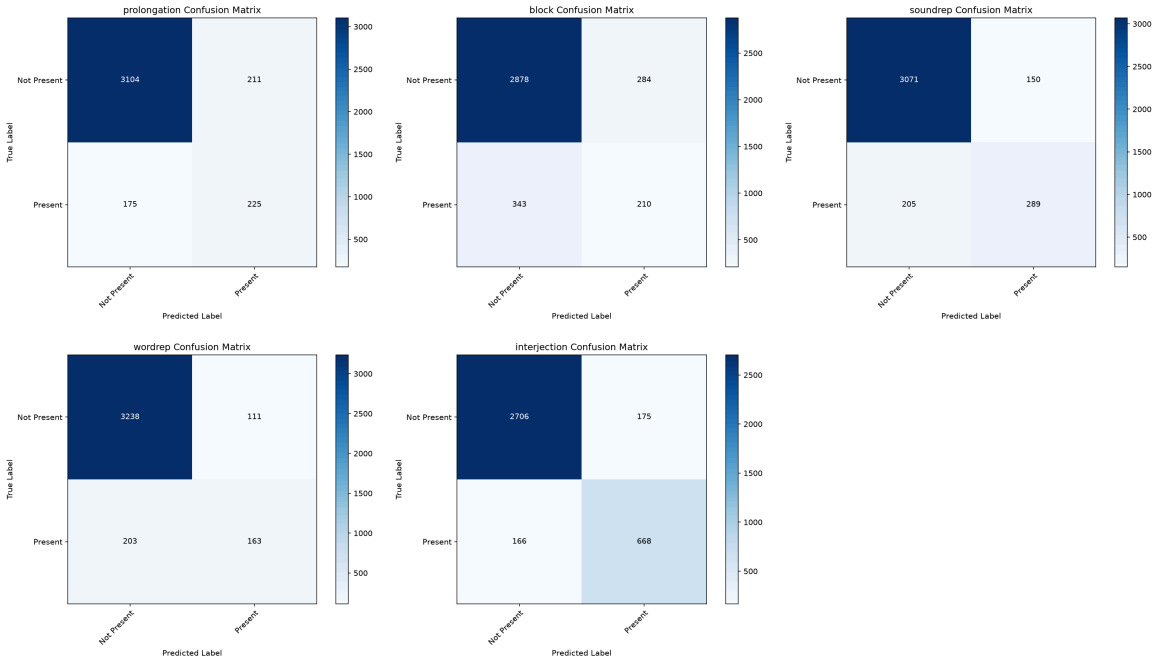


Figure 5.3: Confusion matrices for the five binary classifiers

Each confusion matrix contrasts the true labels against the predicted labels for one binary classifier, so the diagonal cells count correct decisions and the off-diagonal cells expose the direction of error. A strong diagonal with a light upper-right corner means the classifier rarely over-triggers, while a heavy lower-left corner indicates missed events, and the balance between the two is what separates a conservative detector from an over-sensitive one. Interjection shows the cleanest diagonal, consistent with its high F1 and AUROC,

because its acoustic signature, typically a short vocalized filler, is spectrally distinct from fluent speech. Block and word repetition show more spread, since prolonged silences and repeated words can resemble pauses and normal hesitation, which the classifiers then confuse at the boundary. Reading the matrices alongside the per-class table makes the trade-off explicit: a classifier tuned for high recall shifts mass into the false-positive corner, while one tuned for precision leaves more events undetected. This is precisely why the shipped system keeps separate binary classifiers instead of a single multi-class head, since each event type can be calibrated to its own operating point without disturbing the others.

The Wav2Vec2-based model family also provides the frame-level localizer, evaluated on the same 3,715 test samples at a threshold of 0.5; the shipped registry inference threshold for this localizer is 0.3. Results are in Table 5.2.

Metric	**Value**	**Metric**	**Value**
Frame Precision	0.805	Detection Accuracy	0.586
Frame Recall	0.738	Mean IoU	0.779
Frame F1	0.770	False Alarms/min	4.28
Frame Specificity	0.847	Predicted Events	1,846

Table 5.2: Wav2Vec2 localizer evaluation results

The localizer detects most dysfluency events with good accuracy: frame precision is 0.805 and recall is 0.738, so most true events are found and most predictions are correct. The mean IoU of 0.779 means the regions it finds overlap well with ground truth. Of 995 true events it predicted 1,846, of which 1,263 were false alarms (4.28 per minute).

The gap between frame-level and event-level behaviour is worth emphasizing. A frame F1 of 0.770 means that most individual frames are labelled correctly, but the event count shows that the localizer predicts nearly twice as many events as are actually present, so a clinician using the raw output would see clusters of short, spurious detections around genuine events. The 4.28 false alarms per minute figure is the practical consequence of that over-prediction, and it is the main reason the shipped registry applies a higher inference threshold of 0.3 rather than the 0.5 used for reporting. Raising the threshold trades recall for precision by removing low-confidence detections, which reduces the false-alarm rate at the cost of occasionally dropping a weak but real event. The choice of threshold is therefore a clinical decision as much as a technical one, and the system exposes it as a configurable parameter so that a screening workflow can favour sensitivity while a detailed assessment can favour precision. The following subsection turns to how

the classifiers behind these detections were trained, since the localizer and the classifiers share the same pretrained backbone and the same preprocessing conventions.

These frame-level metrics are visualized in Figure 5.4.

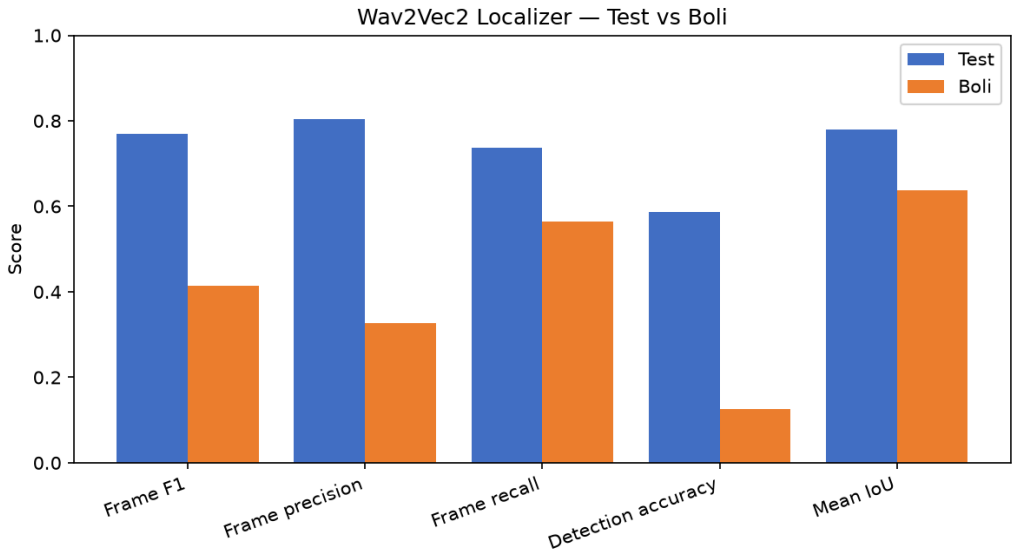


Figure 5.4: Frame-level metrics of the Wav2Vec2 localizer

5.3.2 Wav2Vec2 Classifier Training

The training phase prepares the models that the system relies on for classification and localization.

The system is trained and evaluated on three publicly available stuttering datasets: Project Boli, SEP-28K, and UCLASS. Project Boli is a multilingual Indian language dataset with stuttering data across Hindi, Kannada, Telugu, and other Indian languages, sourced from GitHub, and includes both read and spontaneous speech. The ingestion resolves only the English task codes of this dataset, so the Boli material used here is English speech; it is therefore a cross-corpus rather than a cross-lingual evaluation set. SEP-28K provides roughly 28,000 audio clips from stuttering support group podcasts, annotated for blocks, prolongations, sound repetitions, word repetitions, and interjections, and was obtained from Kaggle. UCLASS is an additional stuttering speech dataset from Kaggle that provides supplementary annotated clips. Each dataset is normalized into a unified format with a combined_labels.csv file of multi-label binary annotations and interval files per clip. The merged dataset is split 80:10:10 into training, validation, and testing subsets, and during training the training subset is re-split 80/20 by a stratified split into a fine-tuning set and a validation set used for early stopping and threshold selection.

Each classifier is trained in a two-stage scheme. The backbone stays frozen for the first three epochs so the classification head can stabilize, then it is unfrozen with a learning

rate scaled to 0.1 of the head learning rate. Focal Loss with a gamma value of 2 handles the class imbalance, and the AdamW optimizer runs at a learning rate of 3×10^{-5} with a weight decay of 0.01 and a 500-step warm-up. Early stopping with a patience of 5 epochs prevents overfitting, and the checkpoint with the highest macro F1-score on the validation set is saved as the final model. During training, data augmentation applies waveform-level transforms (random noise injection, time stretching, pitch shifting, temporal shifting, and amplitude scaling) and spectrogram-level transforms (time and frequency masking). Checkpoints are stored under names that encode their training hyperparameters through a fingerprint convention, with checkpoint-based resume, an audio preprocessing cache for fast re-runs, `torch.compile` for roughly twice the training speed on CUDA, and gradient accumulation for effective batch size scaling.

Several problems encountered during training were resolved. BCEWithLogitsLoss with a single logit per class created a zero-gradient equilibrium where the model collapsed to always predicting the majority class; switching to CrossEntropyLoss with two logits (present and not present) fixed this. Gradient clipping in the classifier loop silently killed learning by scaling the classifier head gradients down, so it was removed from the classification loop while the localizer loops retained it. Mixed-precision training with a GradScaler produced NaN gradients; using `torch.autocast` alone worked. Freezing the back-bone for 3 epochs let the classifier head stabilize before fine-tuning, and longer freeze durations hurt performance.

5.3.3 Model Evaluation of the CNN-based Models

The CNN-based models form the second model family, built to run classifications with far fewer parameters and to compare generalization against the pre-trained Wav2Vec2 baseline.

Four CNN-based multitask classification architectures were evaluated: CNN Pool, CNN Single, CNN-LSTM, and CNN-Transformer, ranging from 0.27M to 0.54M parameters. On the held-out test set they score markedly lower than the Wav2Vec2 classifiers, with tuned F1 values between 0.249 and 0.271. On the cross-corpus Boli set, however, the ranking flips: CNN Single reaches 0.476 and CNN-LSTM 0.508, the best cross-corpus result of all seven architectures, suggesting that the lighter models generalize better to unseen speakers.

The CNN spectrogram localizer was evaluated on the same 3,715 test samples at a thresh-old of 0.5. It is far more permissive than the Wav2Vec2 localizer: frame recall is high (0.873) but precision is much lower (0.527), and it predicted 4,483 events against 995 true events, with 4,329 false alarms (23.49 per minute). Its detection accuracy (0.155) and

specificity (0.330) are correspondingly low. The heavier Wav2Vec2 localizer is therefore shipped as the default; the registry inference threshold is 0.3 for both localizers.

5.3.4 CNN-based Model Training

The CNN-based models are trained on the same unified dataset and with the same two-stage scheme used for the Wav2Vec2 classifiers, but they consume mel-spectrograms instead of raw audio embeddings. CNN Pool applies global pooling over the convolutional feature maps, CNN Single processes a single spectrogram path, CNN-LSTM adds a recurrent layer to capture temporal context, and CNN-Transformer uses a transformer encoder over the spectrogram features. Spectrogram-level augmentation, namely time and frequency masking, is applied during training alongside the waveform-level transforms. Because these networks have only 0.27M to 0.54M parameters compared with roughly 94M to 97M for the Wav2Vec2 family, they train quickly and were used to study the trade-off between parameter count and cross-corpus generalization.

5.3.5 Hyper Parameter Tuning

A threshold sweep on the validation set tested thresholds from 0.1 to 0.9 in steps of 0.05 for each class. The threshold that maximized F1 for each class was stored in the model registry under the multitask configuration. The results are in Table 5.3.

Class	Default (0.5) F1	Optimal Threshold	Tuned F1	Improvement
Prolongation	0.495	0.55	0.457	−7.8%
Block	0.364	0.35	0.429	+17.8%
Sound Repetition	0.586	0.40	0.621	+6.0%
Word Repetition	0.486	0.35	0.499	+2.8%
Interjection	0.776	0.45	0.773	−0.5%
Macro Average	**0.542**	–	**0.556**	**+2.6%**

Table 5.3: Threshold optimization results

Threshold tuning raised the macro F1 from 0.542 to 0.556. Block improved the most (0.364 to 0.429) because the default threshold of 0.5 is too high for a class with strong imbalance. For prolongation and interjection the tuned threshold lowers F1 slightly on the test set, but per-class tuning on the validation set still yields a net gain of +2.6% overall.

Additional hyper parameters were tuned during development. The backbone freeze duration was compared at 3 and 20 epochs; freezing for 3 epochs (MT frz3) clearly beats 20 epochs (MT frz20) on the test set, showing that early unfreezing matters for fine-tuning. The learning rate scaling factor of 0.1 between the head and the unfrozen backbone, a

500-step warm-up, a weight decay of 0.01, and gradient accumulation for effective batch size scaling together gave the most stable training runs.

5.3.6 Performance Analysis of the Models

Seven architectures were compared in total, ranging from the shipped five independent Wav2Vec2 classifiers to various CNN-based multitask models. Test set and cross-corpus Boli results are in Table 5.4.

Architecture	**Type**	**Parame-ters**	**Test F1 (0.5)**	**Test F1 (Tuned)**	**Boli F1 (Tuned)**
5x Wav2Vec2	Classifier	94.6M	0.573	0.570	0.292
MT Wav2Vec2 (frz3)	Multitask	97.3M	0.542	0.556	0.238
MT Wav2Vec2 (frz20)	Multitask	97.3M	0.141	0.378	0.214
CNN Pool	Multitask	0.34M	0.149	0.256	0.332
CNN Single	Multitask	0.27M	0.212	0.262	0.476
CNN-LSTM	Multitask	0.46M	0.242	0.249	0.508
CNN-Transformer	Multitask	0.54M	0.269	0.271	0.345

Table 5.4: Comparative study of seven architecture variants

The shipped five-classifier Wav2Vec2 configuration scores highest in distribution (tuned F1 0.570), narrowly ahead of the multitask variant (0.556); both benefit from large pre-trained speech representations. The CNN models have far fewer parameters but score lower on the test set. On the Boli cross-corpus set the ranking flips: CNN-LSTM reaches 0.508, the best cross-corpus result, followed by CNN Single (0.476). Lighter models seem to generalize better to unseen speakers, while heavy pre-trained models do better on familiar data.

The per-class tuned F1 scores on the Boli set are shown in Figure 5.5, where the lighter CNN variants lead the comparison.

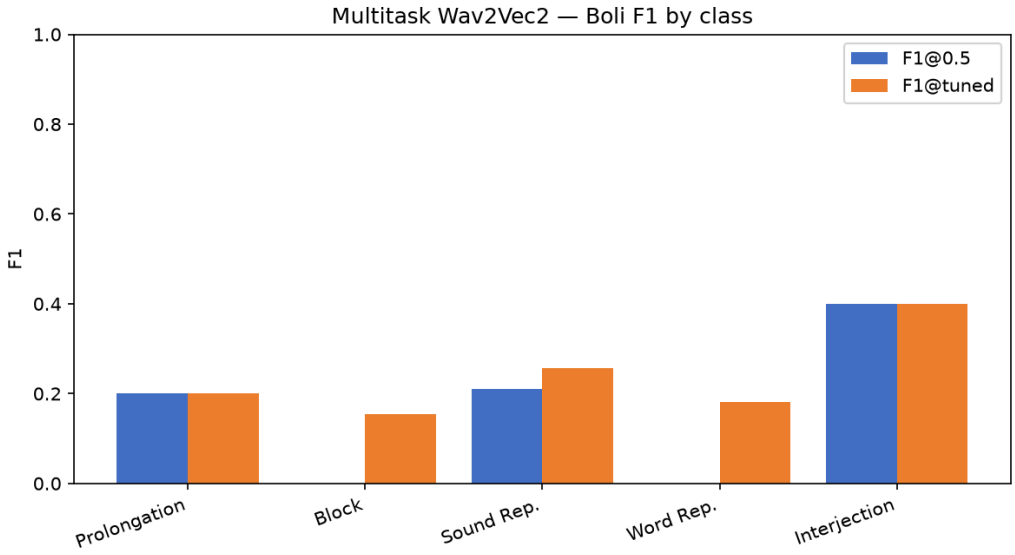


Figure 5.5: Per-class tuned F1 on the Boli set

It was concluded that the Wav2Vec2 family should be shipped for in-distribution accuracy, with the CNN-LSTM variant available as a lighter alternative that transfers best to unseen speakers and languages.

This conclusion reflects a deliberate trade-off rather than a claim that one architecture dominates on every axis. The Wav2Vec2 classifiers benefit from representations learned on large amounts of unlabeled speech, which is why they lead on the test set where the speakers and recording conditions resemble the training distribution. On the Boli corpus, where speakers and languages differ from the podcast-style training data, the advantage narrows and the smaller CNN models transfer better, suggesting that heavy pretrained backbones overfit to the acoustic characteristics of their training corpora. The practical implication is that the system reports both numbers instead of a single headline score, and that deployment should state which regime is expected. For a clinic working with speakers and equipment similar to the training data, the Wav2Vec2 configuration is the better default; for a deployment that must handle unfamiliar speakers or languages without retraining, the CNN-LSTM variant is the safer choice and runs on far more modest hardware.

The comparison also clarifies where further work would pay off: the gap between in-distribution and cross-corpus performance is larger than the gap between architectures, so data coverage and speaker diversity matter more than adding parameters.

CHAPTER 6

SCREENSHOTS

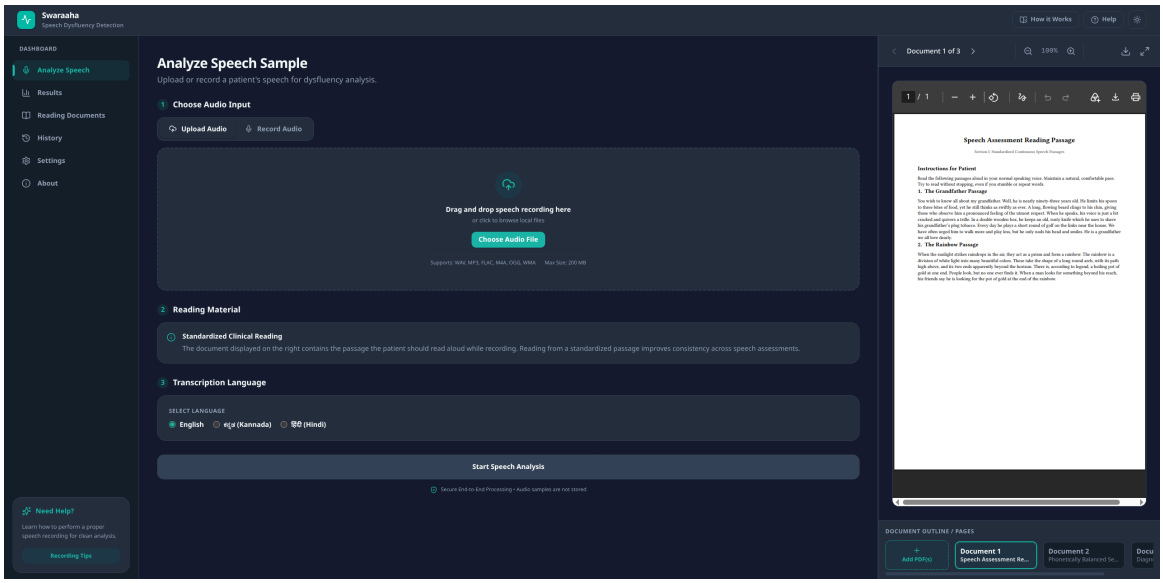


Figure 6.1: Home page of the web interface

The home page of the Swaraaha web application, shown in Figure 6.1, features the Analyze Speech Sample panel, which allows users to upload a supported audio file or record speech directly, select the transcription language, and begin the analysis. A reading-material viewer is provided alongside the input controls so that users can read a standardized passage before recording.

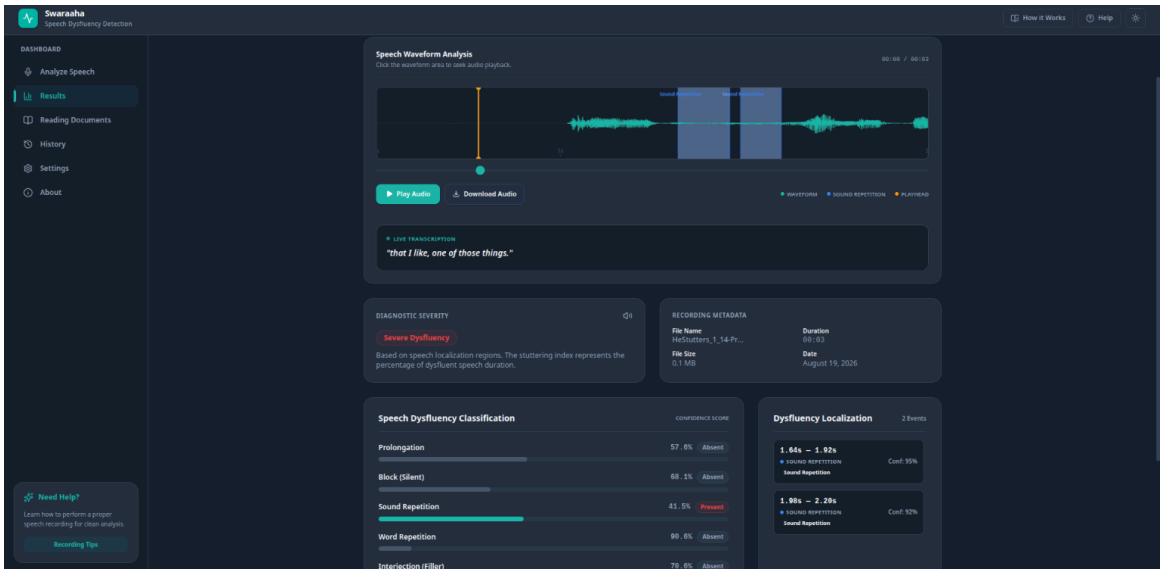


Figure 6.2: Results page of the web interface

The web results page, shown in Figure 6.2, presents the recorded waveform with localized dysfluency regions, synchronized playback controls, a timestamped live transcrip-

tion, and confidence scores for prolongation, block, sound repetition, word repetition, and interjection. Users can upload another sample, review the complete analysis, or download a PDF report.

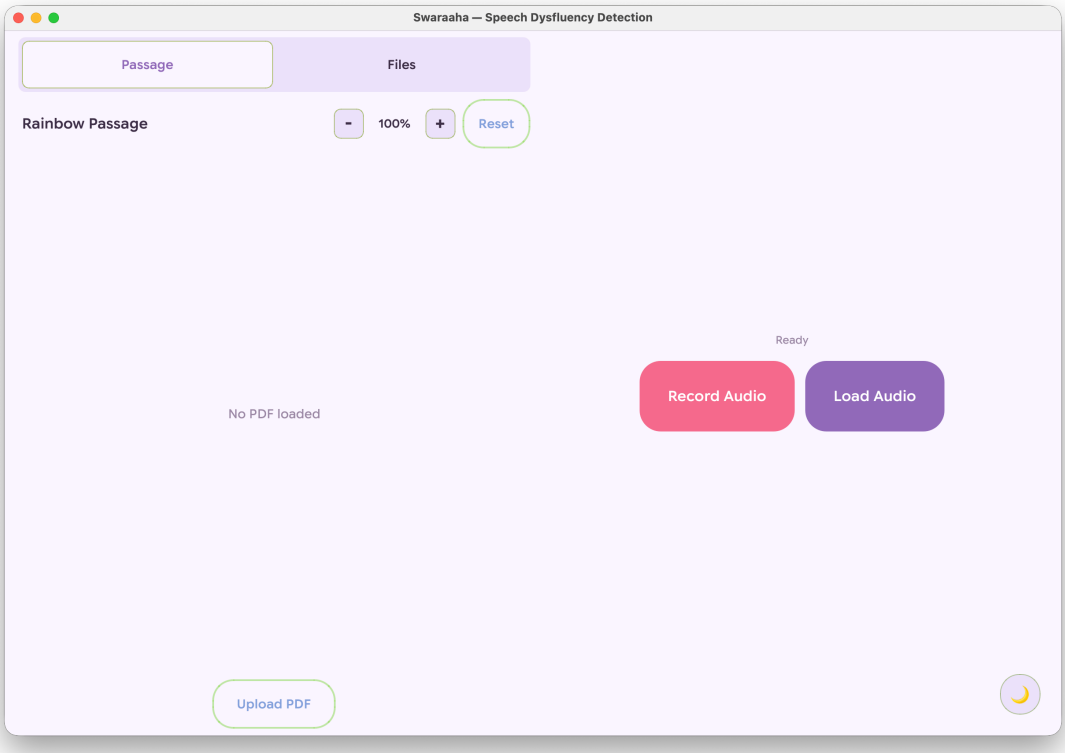


Figure 6.3: Home page of the desktop application

The home page of the desktop application, shown in Figure 6.3, allows users to open a standardized reading passage, load an audio file from the local file system, or record new speech. The interface also displays the current processing status and provides the controls required to continue once an audio sample has been selected. The desktop client mirrors the web workflow but runs entirely on the local machine, so it can be used in settings where a network connection is unreliable or where audio cannot leave the clinic. The reading passage viewer is embedded in the same window as the input controls, which means a speaker can read the standardized text and start recording without switching applications, reducing the variability introduced by different reading setups. The status indicator reports each processing stage in sequence, and the controls remain disabled until a valid audio sample is available, which prevents accidental submissions and makes the interaction predictable for non-technical users. Because the model package is imported directly rather than called over HTTP, the desktop application avoids the overhead of serializing audio and results, and it remains usable when the web backend is not running.

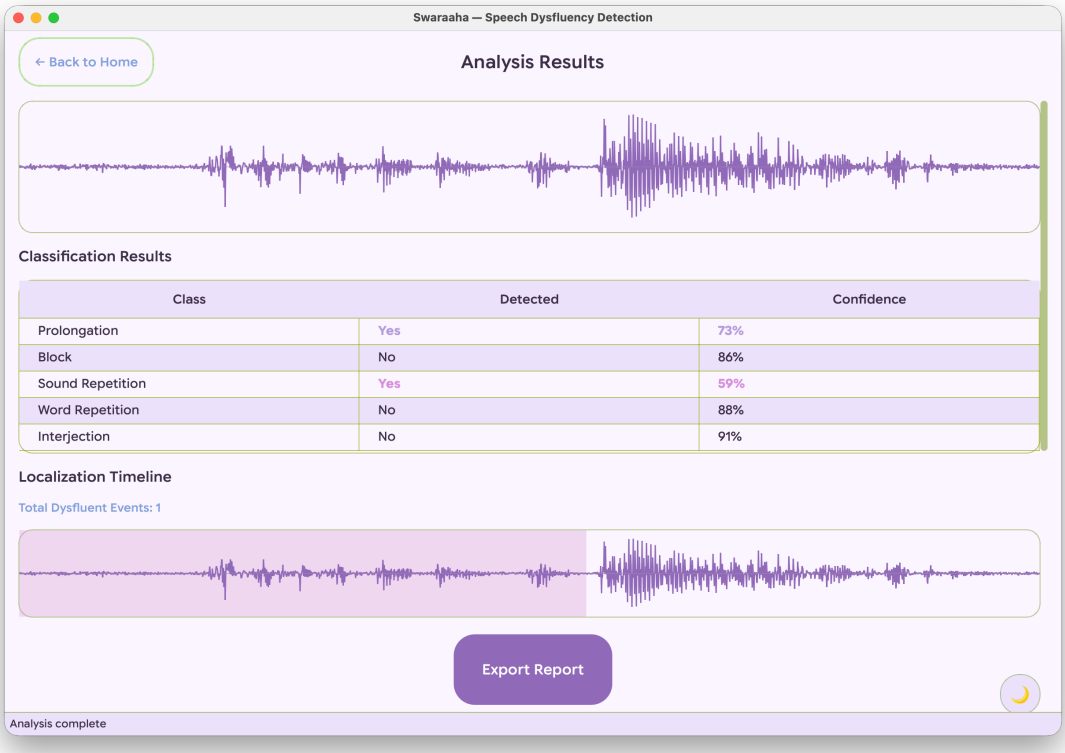


Figure 6.4: Analysis result page of the desktop application

The desktop analysis results page, shown in Figure 6.4, presents a classification table that reports whether each of the five dysfluency types was detected and the corresponding confidence value, while the localization timeline marks detected events on the waveform. The user can review the analysis, return to the home page, or export the results as a PDF report. The results page keeps the classification table and the timeline visually linked: selecting a row highlights the corresponding region on the waveform. This coupling lets a clinician move from a summary judgement to the exact evidence behind it without manually searching the audio, which is the main practical advantage of combining classification and localization in one view. The confidence values are shown per class rather than collapsed into a single score, so a weak detection is visible instead of being hidden. Exporting the PDF captures the table, waveform overlays, and transcript in one document.

CHAPTER 7

CONCLUSION AND SCOPE FOR FUTURE ENHANCEMENTS

7.1 CONCLUSION

The proposed stuttering classification and localization system supports accurate, objective assessment of speech dysfluencies through automated analysis of speech audio. Combining Wav2Vec 2.0 feature extraction with Wav2Vec2 frame-level localization and Whisper transcription strengthens feature extraction and improves classification performance. The pipeline covers data preprocessing, classification of five dysfluency types with five independent Wav2Vec 2.0 binary classifiers, event localization, transcription in English, Kannada, and Hindi, severity estimation, and PDF report generation.

The shipped default of independent classifiers reached a macro F1 score of 0.573, while the multitask shared-backbone variant scored 0.542 at the default threshold and 0.556 after per-class threshold tuning, with interjection the strongest class (F1 = 0.776, AUROC = 0.935) and block the weakest. The Wav2Vec2 localizer produced balanced frame-level results (precision 0.805, recall 0.738, F1 0.770) with a mean IoU of 0.779. Overall, the complete system achieves a detection accuracy of 0.586 for localizing dysfluency events, alongside a frame-level F1 of 0.770 and a macro F1 of up to 0.573 for classification.

The system gives clinicians, speech-language pathologists, and people who stutter consistent predictions that reduce the burden of manual analysis, limit diagnostic variability, and support timely assessment. Classification accuracy for sparse classes such as block and word repetition (macro F1 0.556) is not yet sufficient for a standalone diagnostic tool, and cross-corpus generalization weakens outside podcast-style data (multitask F1 drops from 0.556 on the test set to 0.238 on the Boli corpus). Despite these limitations, the modular architecture combines classification, localization, transcription, and reporting in one pipeline, providing a practical foundation for real-world speech assessment.

7.2 SCOPE FOR FUTURE ENHANCEMENTS

Looking ahead, future enhancements of the stuttering detection system could focus on expanding its functionality and improving detection accuracy. Incorporating larger and more diverse datasets from multiple clinical sources would help the model generalize better across different speakers, stuttering styles, and languages. The integration of additional

deep learning architectures, such as larger pretrained models like Wav2Vec2-large and HuBERT-large, and advanced localization techniques such as curriculum learning, harder negative mining, and boundary-aware loss functions, could further refine the detection and localization of dysfluency events. Moreover, developing a user-friendly web and mobile interface for clinicians could make the system more accessible in real-time therapy settings. The system could also be enhanced with explainable AI features, such as attention visualization and saliency maps, to provide visual insights into how predictions are made, increasing trust and transparency in clinical decision-making. Multilingual models like MMS and XLS-R could extend language coverage beyond English, Kannada, and Hindi, while synthetic dysfluent speech generated with LLMs and text-to-speech systems could address the data scarcity problem during training. Finally, integrating this system with clinic databases and electronic health record systems could enable seamless clinical deployment, supporting doctors and speech-language pathologists in early assessment and personalized treatment planning for stuttering.

REFERENCES

1. P. Arbajian et al., “Effect of Speech Segment Samples Selection in Stutter Block Detection and Remediation,” in Proc. of the International Conference on Speech and Computer, Springer, 2023, pp. 1-10.

2. V. Mitra et al., “Analysis and Tuning of a Voice Assistant System for Dysfluent Speech,” in Proc. Interspeech, 2021, pp. 3695-3699.

3. P. Mohapatra et al., “Speech Disfluency Detection with Contextual Representation and Data Distillation,” arXiv preprint arXiv:2306.05240, 2023.

4. J. Liu et al., “Automatic Speech Disfluency Detection Using Wav2Vec 2.0 for Different Languages with Variable Lengths,” in Proc. IEEE Spoken Language Technology Workshop (SLT), 2023, pp. 1-6.

5. A. Romana et al., “Automatic Disfluency Detection from Untranscribed Speech,” in Proc. Interspeech, 2024, pp. 1-5.

6. D. Wagner, S. P. Bayerl, I. Baumann, K. Riedhammer, E. Nöth, and T. Bocklet, “Large Language Models for Dysfluency Detection in Stuttered Speech,” in Proc. Interspeech, 2024, pp. 1-5.

7. S. A. Sheikh et al., “Advancing Stuttering Detection via Data Augmentation, Class-Balanced Loss and Multi-Contextual Deep Learning,” IEEE Access, vol. 12, 2024.

8. X. Zhou et al., “YOLO-Stutter: End-to-End Region-Wise Speech Dysfluency Detection,” in Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), 2024.

9. X. Zhou et al., “Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection,” in Proc. IEEE Spoken Language Technology Workshop (SLT), 2024, pp. 1039-1046, doi: 10.1109/SLT61566.2024.10832222.

10. J. Zhang et al., “Analysis and Evaluation of Synthetic Data Generation in Speech Dysfluency Detection,” in Proc. Interspeech, 2024.

11. S. Kim and A. Kumar, “FluentNet: End-to-End Detection of Speech Disfluency with Deep Learning,” in Proc. IEEE Spoken Language Technology Workshop (SLT), 2022, pp. 784-790.

12. V. Uloza et al., “An Artificial Intelligence-Based Algorithm for the Assessment of Substitution Voicing,” Journal of Voice, vol. 38, no. 3, pp. 512-520, 2024.

13. H. Müller and C. Lee, “Reinvestigating the Neural Bases Involved in Speech Production of Stutterers: An ALE Meta-Analysis,” Brain and Language, vol. 238, 2023.

14. A. Baevski et al., “Wav2Vec 2.0: A Framework for Self-Supervised Learning of Speech Representations,” in Proc. Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 12449-12460.

15. P. Khanna et al., “StuD: A Multimodal Approach for Stuttering Detection with RAG and Fusion Strategies,” arXiv preprint arXiv:2409.12345, 2024.

16. C. Lea and V. Mitra, “SEP-28K: A Dataset for Stuttering Event Detection from Podcasts,” in Proc. Interspeech, 2023.

17. O. Shonibare et al., “Enhancing ASR for Stuttered Speech with Limited Data Using Detect and Pass,” in Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), 2024.

18. S. P. Bayerl, D. Wagner, E. Nöth, and K. Riedhammer, “Detecting Dysfluencies in Stuttering Therapy Using Wav2Vec 2.0,” in Proc. Interspeech, 2022, pp. 2878-2882.

19. S. Bayerl et al., “Dysfluencies Seldom Come Alone: Detection as a Multi-Label Problem,” in Proc. Int. Conf. Statistical Language and Speech Processing, 2023.

20. R. Gong et al., “AS-70: A Mandarin Stuttered Speech Dataset for Automatic Speech Recognition,” in Proc. IEEE Spoken Language Technology Workshop (SLT), 2024.

21. X. Liu et al., “An End-to-End Stuttering Detection Method Based on Conformer and BiLSTM,” in Proc. IEEE Spoken Language Technology Workshop (SLT), 2024.

22. A. Batra et al., “Boli: A Dataset for Understanding Stuttering Experience,” arXiv preprint arXiv:2407.09876, 2024.

23. A. R. Valente et al., “Clinical Annotations for Automatic Stuttering Severity Assessment,” Journal of Communication Disorders, vol. 105, 2023.

24. T. Grósz et al., “Wav2Vec2-Based Paralinguistic Systems to Recognise Vocalised Emotions and Stuttering,” in Proc. Interspeech, 2023.

25. J. Tang et al., “Speech Annotation Guidelines with People Who Stutter,” Stammering Research, vol. 19, no. 1, 2024.

26. A. Romana et al., “FluencyBank Timestamped: An Updated Data Set for Disfluency Detection and Automatic Intended Speech Recognition,” Language Resources and Evaluation, 2024.

27. L. Nie et al., “MMSD-Net: Towards Multi-Modal Stuttering Detection,” in Proc. ACM Multimedia, 2024.

28. R. P. Buzzeti et al., “Detecting Stuttering with Artificial Intelligence: A Hybrid Method for Brazilian Portuguese,” Speech Communication, vol. 158, 2024.

29. FastAPI, “FastAPI: Modern, Fast Web Framework for Building APIs with Python,” 2024. [Online]. Available: https://fastapi.tiangolo.com

30. Hugging Face, “Transformers: State-of-the-Art Machine Learning for PyTorch, TensorFlow, and JAX,” 2024. [Online]. Available: https://huggingface.co/docs/transformers

31. React, “React: A JavaScript Library for Building User Interfaces,” Meta, 2024. [Online]. Available: https://react.dev

PERSONAL PROFILE

	**Prof. Ajay Shastry C G** • An educator with eight years of experience, his research and teaching interests center around Cybersecurity, AI/ML, and the Internet of Things. • Completed B.E in Computer Science at Vivekananda College of Engineering & Technology, Puttur from 2012 - 2016 and M. Tech in Computer Science at Sahyadri College of Engineering and Management, Mangalore from 2016 - 2018. Email ID: ajayshastrycg.ai@vcetputtur.ac.in Phone: +91 99021 94905
	**K Shreekrishna Upadhyaya** USN: 4VP23AI020 Email ID: upadhyayashreekrishna@gmail.com Phone: +91 96861 08613
	**M Chethan Keshav Bhat** USN: 4VP23AI023 Email ID: chethanbhat777@gmail.com Phone: +91 80734 68867
	**Skanda Prasad K** USN: 4VP23AI051 Email ID: skandaprasad02@gmail.com Phone: +91 87628 89622



Srinivas Hegde M

USN: 4VP23AI054

Email ID: hegdesrinivasm@gmail.com

Phone: +91 90601 59605